SE(3)-WAM: Learning Dynamic Manipulation from Forecasts at Action Time

Anonymous CVPR submission

Figure placeholder: Fig. 1 teaser — learning to act on the future 4D scene (real robot)

Layout: full width, two parts. **(a) Left, ∼62%:** one real-robot frame of the *conveyor grasp* at the instant a chunk is issued. Overlays in the paper's semantic colors: blue = the target's past SE(3) poses from the causal tracker (small triads at $-350 \ldots 0$ ms, fading with age); orange = target SE(3) triads at the commands' application times, the one at the first application time τ^{a} emphasized, taken from the training-time world head evaluated offline; light orange arrows = coarse 3D flow on the context on the 14×14 grid only; purple = the commanded gripper path meeting the target at τ^{a}. A gray ghost marks the pose observed at capture, with a dashed red arrow showing where a reactive policy would close. A thin timeline strip under the image: capture $\tau_t \rightarrow$ latency $\ell_t \rightarrow$ application τ^{a}. **(b) Right, ∼38%:** top: "Image-space WAM: future frames or latents" (filmstrip icon); middle: "SE(3)-WAM: SE(3) for the target, 3D flow for the context, indexed by application time" (triads + sparse arrows); bottom **(required)**: SR vs. added latency (0–200 ms) for SE(3)-WAM and the strongest baseline (data: the latency sweep of Appendix G.2). **Rules:** real photo kept unretouched; overlays drawn from logged tracker/forecast/command data of that trial; caption states the trial's outcome.

Figure 1. **Forecasting the scene at action time.** Between image capture and the first command, a part on the conveyor moves farther than the grasp tolerates. SE(3)-WAM encodes the target's metric motion (blue) and is trained to forecast the target's SE(3) pose at each command's application time, together with a coarse 3D flow of the surrounding scene (orange, predicted only during training), so that its commands meet the target there (purple). Image-space world–action models forecast appearance instead. Right: success against added latency on Dynamic-14.

Abstract

Manipulating moving objects requires commands that fit the scene at the moment they take effect, a few hundred milliseconds after the image was captured. We analyze this delay with an error budget in physical time, in which the target's contact error grows with its speed times the lead, while the rest of the scene only needs to be predicted to within its clearance. The budget suggests what a policy should remember (the target's metric motion), what it should predict and how precisely (an SE(3) trajectory for the target and a coarse 3D flow for the context), and for which instants (the commands' application times). We build SE(3)-WAM, a world–action transformer that follows these choices and learns lead-conditioned commands together with this 4D forecast. On Dynamic-14, fourteen dynamic tasks that we evaluate in real time, SE(3)-WAM succeeds in [XX.X]% of held-out episodes against [XX.X]% for the strongest baseline. Supervising the forecast at the commands' application times gives the benefit of prediction without generating a forecast at test time, so that deployment runs only the action stream. A base pretrained on DROID with automatically lifted 4D labels transfers to a real conveyor grasp, where SE(3)-WAM succeeds in [XX]% of 180 trials against [XX]% for $\pi_{0.5}$.

1. Introduction

Picking a part from a conveyor, placing an object into a moving bin, and catching a ball that rolls off a ramp all require a command that fits the scene at the moment it takes effect. Between image capture and gripper closure, the policy runs inference, commands wait in a queue, and the arm moves. A part on a conveyor at 0.10 m/s travels 15 mm during a 150 ms delay, which is more than the [X] mm that a parallel-jaw grasp of it tolerates. Most state-of-the-art visuomotor policies, from diffusion policies to vision–language–action (VLA) and world–action models (WAMs) [10, 31, 42, 49], were developed for quasi-static scenes, where this delay hardly matters [1, 12, 46].

Recent work adds temporal context in three ways. Policies receive stacked frames, optical flow, or compressed video tokens as history [23, 27, 29, 46, 59], world models forecast future video, latents, object states, or velocity fields [12, 35, 49, 55], and execution schemes keep commands timely despite inference delay [6, 36, 39]. It remains unclear which information a dynamic policy needs, how precisely, and for which instant. We argue that the answer follows from where interaction errors come from. The error of a grasp or a placement is set by the target's metric pose at the moment the command applies, and the rest of the scene matters only through the clearance it leaves. Pixel histories leave this motion implicit, and image-space forecasts spend

most of their capacity on appearance.

We make this argument concrete with an error budget in physical time (Sec. 3). It splits the contact error into localization, motion, timing, unmodeled-dynamics, and rotation terms, and the rest of the scene matters only where its prediction error exceeds the clearance. The algebra is elementary and latency compensation is classical in control and tracking [3, 4, 34], but for learned policies the budget gives a way to allocate precision across the scene and predictions that we can test, such as the lead beyond which a policy that ignores motion must fail. It also suggests three design choices. A policy should *remember* the target's metric motion, *predict* the target's SE(3) trajectory and a 3D flow of the context no finer than its clearance requires, and *act* on forecasts and commands indexed by their application time.

We build SE(3)-WAM around these choices (Fig. 3). A transformer encodes the current RGB-D frame, the robot state, and the target's SE(3) history from a causal pose tracker. A world stream predicts the target's SE(3) trajectory and a coarse 3D scene flow at physical horizons, and an action stream generates commands that are conditioned on their lead, the time from image capture to their application. The world stream supervises the shared representation during training, and deployment runs the action stream alone. For the real robot, we lift DROID [20] to metric 4D labels with off-the-shelf perception [17, 18, 32] and pretrain a base model on it.

We evaluate on Dynamic-14, fourteen simulated tasks that we built so that each term of the budget can be varied on its own, on DOMINO [12] and LIBERO [25], and on a real conveyor, in a real-time protocol in which the scene keeps moving while each method computes. SE(3)-WAM reaches [XX.X]% success on held-out Dynamic-14 scenes, [X.X] points above the strongest baseline, although a larger model is ahead when physics pauses during inference. The SE(3) history helps beyond the critical lead, whereas RGB, flow, or video-latent histories add little. Most of this gain survives when the forecast is dropped at deployment, as long as it was supervised at the times the commands apply, much as [51] found for static scenes. The budget's predictions hold approximately, with the largest deviations on the capture tasks[, and one task remains unsolved by every method]. In summary, we contribute

- a simple error budget in physical time that motivates the design and whose predictions we test,
- SE(3)-WAM, which learns lead-conditioned actions together with a role-dependent 4D forecast, and a real-robot base pretrained on DROID with lifted 4D labels, and
- Dynamic-14 with a real-time protocol, on which SE(3)-WAM improves over prior methods. We will release it with the DROID labels and our code.

2. Related Work

Dynamic manipulation. Compensating delay by prediction is classical in control [34] and predictive tracking [3]. Robotic catching and hand–eye systems estimate an object's trajectory from vision, usually with Kalman-type filters [4], and intercept it [2, 5, 15]. Visual servoing closes the loop on image features [19], and dynamic grasping couples motion prediction with reachability [1]. Among learned policies, DOMINO introduces a dynamic benchmark together with PUMA, which reads historical flow and is supervised with future object features [12]. DynamicVLA streams actions and drops stale ones [46], DynamicWAM conditions a WAM on flow and kinematic descriptors [29], and others predict salient future tokens [35] or event timing [56]. Several recent preprints pursue related ideas with motion-aware world models, velocity fields, or synthesized dynamic demonstrations [9, 22, 52]. For execution, real-time chunking keeps control smooth under delay [6], VLASH rolls the robot state forward to the execution time [36], and the concurrent Chronos Policy conditions on observation and execution timestamps [39]. These methods handle time on the robot side or learn object motion from pixels, whereas we ask which scene quantities a policy must know at execution time.

World–action models. WAMs couple future prediction with action generation through predictive features [16], visual subgoals [57], or joint video–action generation [21, 42, 49, 61]. Others add depth, semantics, or dynamic regions to the forecast (e.g., [50, 55]) or compress long histories [27]. Fast-WAM finds that test-time imagination can be skipped without loss on its static tasks [51]. These models forecast in image or latent space, without the metric, object-level state that the error budget points to. SE(3)-WAM forecasts such a state at physical horizons, and we find that in dynamic scenes, too, its benefit survives without test-time imagination when the forecast is supervised at the commands' application times.

Structured 3D and 4D motion. Rigid-body structure has long been used in learned dynamics models and scene flow [7, 8, 37, 48], and 3D point tracking follows pixels through space [45]. Point tracks and 3D flow have served as policy interfaces [44, 47, 60], point clouds as policy inputs [53], and object poses as structure for world models and policies [11, 28]. PointWorld predicts action-conditioned 3D point flow at scale [17], and feed-forward models reconstruct scene geometry [40] and dynamic point tracks [54]. We use rigid structure only for the target and keep the rest of the scene as a coarse 3D flow, both indexed by the commands' application times.

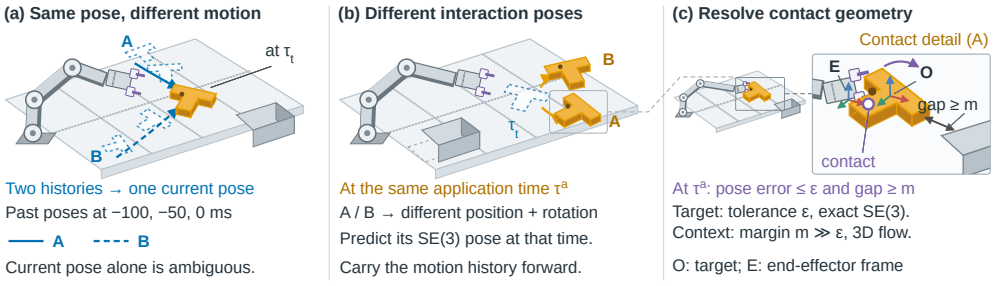


Figure 2. **Three consequences of acting in physical time** (schematic). (a) Two histories reach the same pose at capture time τ_t but imply different motions. (b) At the same application time τ^{a}, they lead to different target poses, in translation and in rotation. (c) A contact requires the end-effector frame E to be aligned with the target O within ε, whereas the context matters only through the gap it leaves, with a margin $m \gg \varepsilon$.

3. Interaction in Physical Time

Physical time. We consider a policy that receives images and robot states together with their capture times. At query t, its history is $\mathcal{H}_t = \{(o_i, s_i, \tau_i)\}_{\tau_i \le \tau_t}$, and it outputs a chunk $A_t = (\mathbf{a}_{t,0}, \dots, \mathbf{a}_{t,H-1})$ whose j-th command takes effect at

$$\tau^{\mathrm{a}}_{t,j} = \tau_t + \ell_t + j\Delta_a, \tag{1}$$

where ℓ_t is the latency from capture to the first application (preprocessing, inference, queuing, and actuation) and Δ_a is the control period. We call $\Delta_{t,j} = \tau^{\mathrm{a}}_{t,j} - \tau_t$ the *lead* of command j, which must fit the scene at $\tau^{\mathrm{a}}_{t,j}$ rather than at τ_t.

Target and context. Let $W_t(\mathbf{x}, \tau) \in \mathbb{R}^3$ be the displacement between τ_t and τ of the scene point that is at $\mathbf{x}$ at time τ_t, expressed in the robot base frame. We split the scene by the precision that the task requires of it (Fig. 2). The *target* $\mathcal{O}$ is the rigid body whose pose error enters a contact or containment tolerance ε, e.g., an object to grasp or a receptacle to place into. The *context* $\mathcal{C}$ contains the bodies that only need to be avoided—a belt, a wire, an occluder—with clearance m. On the target, the motion field is set by its pose $T(\tau) = (R(\tau), \mathbf{p}(\tau)) \in \mathrm{SE}(3)$:

$$W_t(\mathbf{x}, \tau) = T(\tau)\, T(\tau_t)^{-1}\mathbf{x} - \mathbf{x}, \qquad \mathbf{x} \in \mathcal{O}. \tag{2}$$

An interaction error budget. Consider a contact point at lever arm r from the target origin and a policy that acts, explicitly or implicitly, on an estimate $\hat{T}$ of the target pose at τ^{a}. With a rotation error θ, the contact error satisfies

$$e_{\mathrm{c}} \le \|\hat{\mathbf{p}}(\tau^{\mathrm{a}}) - \mathbf{p}(\tau^{\mathrm{a}})\| + 2r\sin(\theta/2). \tag{3}$$

We write the true motion over the lead $\Delta = \tau^{\mathrm{a}} - \tau_t$ as $\mathbf{p}(\tau^{\mathrm{a}}) = \mathbf{p}(\tau_t) + \mathbf{v}\Delta + \boldsymbol{\rho}$, where $\boldsymbol{\rho}$ is what constant-velocity extrapolation misses (acceleration, rolling, impacts), and the estimate as $\hat{\mathbf{p}}(\tau^{\mathrm{a}}) = \hat{\mathbf{p}}(\tau_t) + \hat{\mathbf{v}}\hat{\Delta} + \hat{\boldsymbol{\rho}}$. Adding and subtracting $\mathbf{v}\hat{\Delta}$ gives

$$\|\hat{\mathbf{p}} - \mathbf{p}\| \le \underbrace{e_{\mathrm{loc}}}_{\text{now}} + \underbrace{\hat{\Delta}\, e_v}_{\text{lead}\times\text{motion}} + \underbrace{\|\mathbf{v}\|\, e_\tau}_{\text{speed}\times\text{timing}} + \underbrace{e_{\mathrm{dyn}}}_{\text{dynamics}}, \tag{4}$$

with $e_{\mathrm{loc}} = \|\hat{\mathbf{p}}(\tau_t) - \mathbf{p}(\tau_t)\|$, $e_v = \|\hat{\mathbf{v}} - \mathbf{v}\|$, $e_\tau = |\hat{\Delta} - \Delta|$, and $e_{\mathrm{dyn}} = \|\hat{\boldsymbol{\rho}} - \boldsymbol{\rho}\|$. Rotation decomposes in the same way (Appendix A). The context enters differently. If the commanded path keeps a clearance m from the predicted context positions $\hat{\mathbf{y}}_t(\mathbf{x}, \tau)$, a collision requires a prediction error of at least m at some point and time in the set $\mathcal{P}$ that the path passes:

$$\text{collision} \;\Rightarrow\; \max_{(\mathbf{x},\tau)\in\mathcal{P}} \big\|\hat{\mathbf{y}}_t(\mathbf{x}, \tau) - \mathbf{y}_t(\mathbf{x}, \tau)\big\| \ge m. \tag{5}$$

A trial succeeds if $e_{\mathrm{c}} \le \varepsilon$ and no collision occurs. Eqs. (3)–(5) follow from an identity and the triangle inequality. We use them as a budget rather than as a model of success.

Predictions. Given each task's tolerance and speeds (Appendix B.1): (i) a policy that ignores motion incurs an error of $\|\mathbf{v}\|\Delta$, so it should fail beyond the critical lead $\ell^\star = \varepsilon/\|\mathbf{v}\|$, and its success should depend on the task, speed, and latency only through $\|\mathbf{v}\|\Delta/\varepsilon$. (ii) A least-squares fit to n poses with noise σ over a span T estimates the velocity with an error of about $\sigma\sqrt{12/n}/T$ [4], so a pose history remains useful up to a noise level of $\sigma^\star = \varepsilon T\sqrt{n}/(\Delta\sqrt{12})$. (iii) A forecast that is misaligned with the application times by δ adds $\|\mathbf{v}\|\delta$, which grows with speed and vanishes for stationary targets. (iv) A context forecast with path error e_F is sufficient as long as $m > e_F$.

Design implications. Table 1 maps each term to the part of a policy that can reduce it. *Remember.* Given a reliable current pose, a history reduces e_v and, through acceleration, part of e_{dyn}. Because e_v is multiplied by the lead, the history matters through the target's metric motion state, and pixels, flow, or video tokens help only if a policy recovers this state

Table 1. **Predictions of the error budget.** Each row concerns a term of Eqs. (4)–(5), and Sec. 5.3 tests it on held-out Dynamic-14 scenes by sweeping the named factor. $\ell^\star$ and $\sigma^\star$ are the critical lead and noise level, and e_F is the path error of the context flow.

Principle	Term	Prediction
All	e_c/ε	Without compensation, success depends on task, speed, and latency only through $\|\mathbf{v}\|\Delta/\varepsilon$ and drops once it exceeds about 1.
Remember	$\hat{\Delta}e_v$	SE(3) history helps beyond $\ell^\star$ ([X]–[X] ms across tasks), increasingly with latency and speed.
	$e_v(\sigma)$	Its gain persists up to pose noise $\sigma^\star$ ([X]–[X] mm).
Predict	e_{dyn}, $r\theta$	Learned SE(3) futures beat extrapolation on non-constant motion and point tracks under rotation.
	m	Coarse flow suffices down to clearance e_F.
Act	$\|\mathbf{v}\|e_\tau$	A 50 ms horizon shift costs 5 mm at 0.10 m/s and nothing for static targets. Leads in control steps fail when the latency changes.

from them. *Predict.* Reducing e_{dyn} requires a model of how the target will move, and rotation errors are amplified by r. The target's future should therefore be a full SE(3) trajectory, which by Eq. (2) determines the motion of all its points. Context errors are tolerated up to m, so a 3D flow with an error below m suffices, and appearance does not enter the budget. *Act.* Every term is evaluated at τ^{a}, which depends on the latency. Forecasts and commands should therefore be indexed by physical time, with each command's lead known to the policy. Whether the action should read a forecast online or be trained to encode it is left open by the budget, and we answer this question experimentally (Sec. 5.3).

4. SE(3)-WAM

SE(3)-WAM (Fig. 3) is a transformer with three groups of tokens. Condition tokens encode the observation and the target's motion history (*Remember*), world queries forecast the motion field at physical horizons (*Predict*), and action tokens generate lead-conditioned commands (*Act*). We predict the world deterministically and generate actions with flow matching [24].

4.1. Remember: Metric Target-Motion History

We estimate the target pose in every RGB-D frame at full resolution with a causal tracker. SAM 2 propagates a mask from a prompt on the target [32], and FoundationPose tracks the object from a CAD model or a few reference views [43]. Without either, we use a monocular 3D detector that estimates a metric box pose in every frame [18]. Each estimate is emitted once, with its capture time and a validity flag, and later frames do not revise it. We resample this causal track at $-350, -300, \ldots, 0$ ms, so that its meaning does not depend on the camera rate, and build 8 motion tokens with interval motions, finite-difference velocities and accelerations, and the current pose, in the base frame and in physical units (Appendix C.1). A frozen MAE ViT-B/16 [14] encodes the current RGB image into 196 patch tokens, and the depth image gives each patch a metric 3D position for rotary attention. Robot tokens hold 8 end-effector states and the pose commanded for the first application time, and the model receives no past images. We train on the same causal pose estimates that the policy sees at test time, tracking failures included.

4.2. Predict: A Role-Dependent Motion Field

The world queries forecast W_t at horizons $\tau_t + \delta_h$, with $\delta_h = h{\cdot}50$ ms and $h = 1, \ldots, 8$. These horizons cover the leads of all commands that are executed before the next replan (Appendix A.4). We parameterize the target through Eq. (2). For each horizon,

$$\mathbf{d}_h = \mathbf{p}(\tau_t{+}\delta_h) - \mathbf{p}(\tau_t), \quad \boldsymbol{\phi}_h = \mathrm{Log}\big(R(\tau_t{+}\delta_h)R(\tau_t)^\top\big)^\vee. \tag{6}$$

An object query regresses $(\mathbf{d}_h, \boldsymbol{\phi}_h)$ with a squared translation error plus λ_R times the squared geodesic rotation error ($\mathcal{L}_P$). For rotationally symmetric targets, we exclude the unobservable rotation about the symmetry axis. We represent the rest of the scene as a 3D scene flow, the base-frame displacements $F_h \in \mathbb{R}^{N\times 3}$ of $N = 196$ points on a fixed 14×14 image grid, which are about [X] mm apart in the workspace. One scene query per point, with an embedding of its role (target, robot, or context), regresses the point's displacements and visibility ($\mathcal{L}_F$). The grid is deliberately coarse, as the budget allows, and Sec. 5.3 tests where this is sufficient. Since appearance does not enter the budget, future depth, DINOv3 features [33], and the RGB of moving surfaces serve only as additional training targets for the scene queries ($\mathcal{L}_{\mathrm{aux}}$). All labels are cumulative from the current state, and losses use valid labels only.

4.3. Act: Lead-Conditioned Actions

A chunk holds 8 commands that are applied at the times of Eq. (1). Each command is a base-frame translation and a left-relative rotation of the end effector, plus a gripper command. We take increments relative to the pose commanded for the preceding application time, which the controller knows, instead of the arm's pose at capture [36]. Each action token embeds its lead $\Delta_{t,j} = \ell_t + j\Delta_a$. During training, we sample $\ell \sim \mathcal{U}[0, 250\,ms]$ and read the command labels from the demonstration at $\tau_t + \ell + j\Delta_a$, so that a single model serves any latency in this range. For a normalized chunk $\mathbf{a}_1$, noise $\mathbf{a}_0 \sim \mathcal{N}(0, I)$, and $u \sim \mathcal{U}[0, 1]$, the action loss is

$$\mathcal{L}_A = \mathbb{E}\left\|v_\psi(\mathbf{a}_u, u \mid \mathcal{Z}_t, \hat{\mathcal{W}}_t, \mathcal{G}) - (\mathbf{a}_1 - \mathbf{a}_0)\right\|^2, \tag{7}$$

with $\mathbf{a}_u = (1-u)\mathbf{a}_0 + u\mathbf{a}_1$, condition tokens $\mathcal{Z}_t$, and the world $\hat{\mathcal{W}}_t$ to the extent that the coupling exposes it. The total loss is $\mathcal{L} = \mathcal{L}_A + \lambda_F\mathcal{L}_F + \lambda_P\mathcal{L}_P + \lambda_{\mathrm{aux}}\mathcal{L}_{\mathrm{aux}}$.

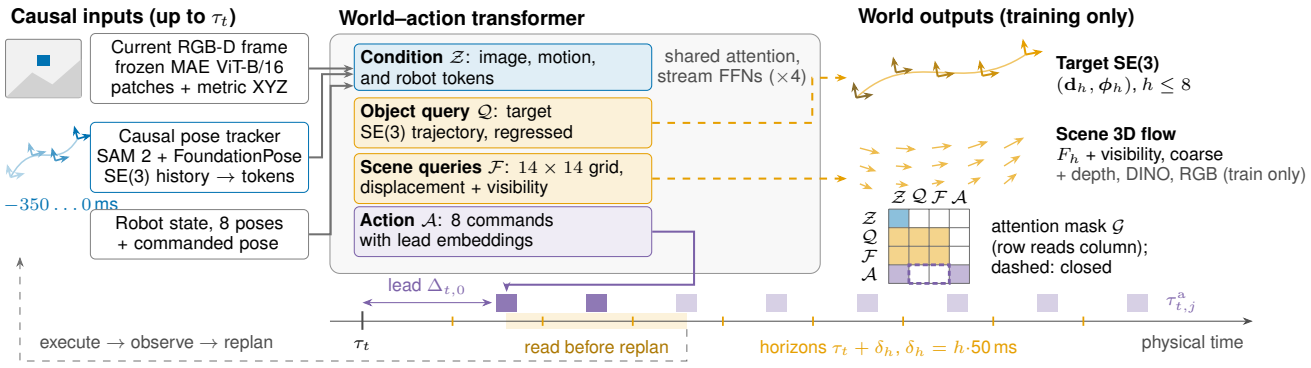


Figure 3. **SE(3)-WAM.** Condition tokens $\mathcal{Z}$ encode the current RGB-D frame, the target's SE(3) history from a causal tracker, and the robot state (*Remember*). An object query $\mathcal{Q}$ and scene queries $\mathcal{F}$ regress the target's SE(3) trajectory and a coarse 3D scene flow with visibility at physical horizons $\tau_t + \delta_h$, and appearance heads add training targets (*Predict*). Action tokens $\mathcal{A}$ generate lead-conditioned commands that apply at $\tau^{a}_{t,j}$, with replanning every 100 ms (*Act*). The mask $\mathcal{G}$ sets which groups each group attends to. Action tokens attend to no world token, so deployment runs the action stream alone.

The attention mask $\mathcal{G}$ acts on four token groups: the condition $\mathcal{Z}$, the object query $\mathcal{Q}$, the scene queries $\mathcal{F}$, and the actions $\mathcal{A}$. Condition tokens attend only to each other, world queries attend to the condition and to each other, and action tokens attend to the condition, to themselves, and to the part of the world that their coupling allows. In Sec. 5.3, we compare couplings in which the action reads nothing from the world, the hidden states of the world queries, or the decoded geometry, and one in which the world also reads the action. In SE(3)-WAM, the action reads no world token. The world losses shape the shared parameters during training, and deployment runs only the action stream. The two streams share the attention projections and have separate feed-forward networks.

Execution and implementation. At test time, we set the leads from the executor's latency budget, the recent p95 inference time plus the actuation delay. The transformer has 4 layers of width 768 with 8 heads, 47.3M backbone parameters, and 62.2M trainable parameters in total. We keep the image encoder and the DINOv3 teacher frozen. On Dynamic-14, we train for 4 epochs (about 40k updates at a global batch size of 16) on eight RTX PRO 6000 GPUs, which takes about 7.7 hours per training seed. At test time, the action stream integrates 5 Euler steps in half precision, with a p95 time-to-action of [XX] ms on an RTX 5090 (Appendix C.6).

4.4. Pretraining on Real Robot Data Lifted to 4D

Real robot data provide neither object poses nor scene flow, so we lift DROID [20] to metric 4D labels without object models. The scene flow comes from the 3D tracks and optimized camera calibrations that PointWorld released for DROID [17]. We take the object named in the instruction as the target. An open-vocabulary detector finds it [26], SAM 2 tracks its mask, and the monocular 3D detector estimates its metric pose. Inputs come from a forward track, as at deployment, and the labels of Eq. (6) from a smoothed forward–backward track over the whole episode. DROID's targets move mostly when the robot moves them ([X]% of windows show target motion before contact), so pretraining mainly teaches metric geometry, 3D flow, and their coupling with lead-conditioned actions. We initialize the policy of each real-robot task from this base. The simulation model is trained from the generic initialization, and a controlled comparison post-trains the base on Dynamic-14 (Appendix E).

5. Experiments

5.1. Setup

Dynamic-14. We built fourteen tasks in MuJoCo/robosuite [38, 62] in five families: grasping a moving target (harmonic, conveyor, translating, rotating, and occluded motion), placing into a moving receptacle, passing a wand along a rotating wire, capturing rolling or sliding objects, and grasping after an impact, each exercising particular terms of the budget (Appendix B.1, Fig. S1). Sweeps vary the speed (0.5–1.5×), the added latency (0–200 ms), the passage clearance ([values] mm), and the pose noise. Scripted experts with simulator state provide 800 demonstrations per task. We select configurations on development scenes and report every Dynamic-14 number on 1,400 held-out scenes (100 per task, new seeds). Our variants average three training seeds, and each baseline is trained once. We report seed means and give paired 95% intervals from a bootstrap over held-out scenes.

Protocol. All methods run in real time under the same executor, and physics advances while a method computes, by its measured latency on one RTX 5090. A new chunk is requested every 100 ms, or when the previous inference

ends, commands execute at their application times, and stale commands are dropped. The +150 ms setting adds 150 ms of delay for every method, and Appendix G.2 compares this protocol with three alternatives. We report the task-balanced success rate (SR), the contact-time error e_t, the position error at first contact e_p, and the p95 time-to-action. On DOMINO [12], we evaluate single-arm interception and tracking, and LIBERO [25] checks that the structured world does not cost performance on static tasks.

Baselines. We compare with reactive policies (ACT [58], Diffusion Policy [10], and $\pi_{0.5}$ [31]), dynamic VLAs (DynamicVLA [46] and PUMA [12]), image- or latent-space WAMs (DreamVLA [55] and OpenWAM-α [42]), and a dynamic WAM (DynamicWAM [29]). Each baseline uses its official code, inputs, and recipe, is trained once on the same demonstrations, and receives a rendered wrist camera if its recipe uses one (Appendix F). SE(3)-WAM uses only the agent view, but it also uses depth and the tracked target pose. To separate these inputs from the representation, $\pi_{0.5}$ and DynamicWAM also receive our SE(3) history tokens (+SE(3)), and we report SE(3)-WAM with poses from the monocular 3D detector, which needs no object model. Two baselines use the tracker alone. *CV input* is SE(3)-WAM without world supervision that receives the constant-velocity extrapolation of the tracked pose as input tokens, and a Kalman interceptor plans to the target pose predicted by a constant-acceleration Kalman filter [4].

5.2. Main Results

In real time, SE(3)-WAM reaches [XX.X]% SR on held-out Dynamic-14 scenes (Table 2), [X.X] points above the strongest baseline, [name of the strongest baseline], and [X.X] points above the strongest baseline that receives the same SE(3) history, [name of the strongest +SE(3) baseline]. With physics paused during inference, however, [name of the strongest paused baseline] leads by [X.X] points. Without the cost of computation, its larger model and richer forecasts pay off, but a query takes it [XXX] ms. The margin of SE(3)-WAM is largest on [rotating targets ($r\theta$) and on fast conveyor and translating targets, where $\|\mathbf{v}\|\Delta$ dominates][, and no method solves the SE(3) placement into the rotating receptacle]. With 150 ms of added delay, SE(3)-WAM keeps [XX.X]%, whereas reactive policies drop to [XX.X]% and image-space WAMs to [XX.X]%. Detector poses instead of a CAD model cost [X.X] points. The CV-input variant and the Kalman interceptor trail SE(3)-WAM by [X.X] and [X.X] points, mostly on non-constant motion, and giving SE(3)-WAM the extrapolated pose as an extra input changes SR by [$\pm$X.X] points (Appendix D.3). On DOMINO, SE(3)-WAM obtains [XX.X]% against [XX.X]% for [name of the strongest DOMINO baseline]. On static LIBERO tasks, it trails $\pi_{0.5}$ by [X.X] points ([XX.X]% vs. [XX.X]%).

5.3. Testing the Predictions

We test each prediction of Table 1 with variants of SE(3)-WAM that change one design axis and keep the data, the training budget, and the other axes fixed (Table 3, Fig. 4, and Appendix D).

Uncompensated motion. We plot the success of every method against $\|\mathbf{v}\|\Delta/\varepsilon$, the target's motion during the lead relative to the tolerance (Fig. 4a). Across tasks, speeds, and latencies, the reactive policies [fall on one curve] (Spearman $\rho = [X.XX]$), and their success halves at [X.X]. The budget gives only a threshold of about one, not the shape of this curve. [The capture tasks scatter most around the curve], because their success also depends on when the gripper closes. Methods that estimate the target's motion shift the curve to the right, SE(3)-WAM the most.

Remember. We hold the current frame, the current target pose, and the robot history fixed. Adding eight frames of RGB, optical flow over the same window with DynamicWAM's motion statistics [29], or video-VAE latents changes SR by [X.X] points (95% interval [[a, b]]). The causal SE(3) history adds [X.X] points and the oracle history [X.X] (Table 3). The gain appears beyond each family's critical lead, grows with latency, and vanishes for stationary targets (Fig. 4b). Frozen readouts show that visual histories carry the motion (Appendix D.2), but at our data scale the policy does not turn it into metric velocity, even with metric XYZ for past frames ([$\pm$X.X] points). Under injected pose noise, the SE(3) history falls to the level of the best visual history at [X] mm (Fig. 4c). This is [X.X] times the model-free prediction $\sigma^\star$, as a learned estimator with a motion prior can be expected to do better, and [X] times the tracker's error.

Predict. Supervising the target's SE(3) trajectory instead of point tracks changes SR by [$\pm$X.X] points overall but adds [X.X] on rotating targets, where the lever arm amplifies rotation errors. Fitting SE(3) to the predicted tracks recovers [X.X] of them. At the application times, the learned SE(3) forecast is as accurate as constant-velocity and constant-acceleration extrapolation of the same history on conveyor motion, but [X] times more accurate on rolling, sliding, and post-impact motion, where e_{dyn} dominates (Fig. 4d). For the context, the 14×14 flow, with a path error of $e_F = [X]$ mm, matches 28^2 and 56^2 grids until the passage clearance falls to [X] mm (Fig. 4e), at [X] times lower training cost for the world stream. Removing the context flow costs [X.X] points, mostly on passage tasks. As the predicted world, future RGB, VAE latents, depth, DINO features, or latents of a frozen video model bring no gain over the SE(3) trajectory and flow ([$\pm$X.X] points at matched capacity), at up to [X] times the

Table 2. **Main results.** Held-out SR (%) on Dynamic-14 (1,400 new scenes, task-balanced, real time, three training seeds for our variants and one for baselines) by family, with 150 ms of added delay, and with physics paused during inference, on single-arm DOMINO and LIBERO (four suites), and p95 time-to-action on one RTX 5090. Inputs: R, RGB and robot state. D, depth. P, target SE(3) history from our tracker with an object model. P′, from a monocular 3D detector. Best in bold. Intervals in Appendix G.

		Dynamic-14, real time unless noted										
Method	Inputs	All	Grasp	Place	Pass	Capture	Impact	+150 ms	Paused	DOMINO	LIBERO	p95 (ms)
ACT [58]	R	—	—	—	—	—	—	—	—	—	—	—
Diffusion Policy [10]	R	—	—	—	—	—	—	—	—	—	—	—
$\pi_{0.5}$ [31]	R	—	—	—	—	—	—	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	R+P	—	—	—	—	—	—	—	—	—	—	—
DynamicVLA [46]	R	—	—	—	—	—	—	—	—	—	—	—
PUMA [12]	R	—	—	—	—	—	—	—	—	—	—	—
DreamVLA [55]	R	—	—	—	—	—	—	—	—	—	—	—
OpenWAM-α [42]	R	—	—	—	—	—	—	—	—	—	—	—
DynamicWAM [29]	R	—	—	—	—	—	—	—	—	—	—	—
DynamicWAM + SE(3)	R+P	—	—	—	—	—	—	—	—	—	—	—
SE(3)-WAM, CV input	RD+P	—	—	—	—	—	—	—	—	—	—	—
Kalman interceptor	D+P	—	—	—	—	—	—	—	—	–	–	—
SE(3)-WAM, detector poses	RD+P′	—	—	—	—	—	—	—	—	—	—	—
SE(3)-WAM	RD+P	—	—	—	—	—	—	—	—	—	—	—

Figure placeholder: Fig. 4 — measured trends against the predictions (six panels)

(a) Uncompensated motion. x: $\|\mathbf{v}\|\Delta/\varepsilon$ at the first command's application time (log); y: SR. One point per method × family × speed × latency; reactive policies (gray) with a fitted logistic curve and Spearman ρ, the predicted break at 1 marked; motion-compensating baselines and SE(3)-WAM (orange) as shifted curves. **(b) Remember / lead.** x: added latency (0–200 ms); y: ΔSR vs. current frame. Lines: RGB window, optical flow, video-VAE latent (gray), SE(3) causal (blue), detector SE(3) (blue dotted), SE(3) oracle (blue dashed); stationary targets as a faint line; ticks at the families' $\ell^\star$. **(c) Remember / noise.** x: injected pose noise σ (mm; symlog axis with $\sigma = 0$); y: SR of SE(3) history; band: best visual history; vertical markers: predicted $\sigma^\star$ and the tracker's measured error. **(d) Predict / target dynamics.** Bars by motion type (constant velocity, harmonic, rolling/sliding, post-impact): forecast error at τ^a of CV, CA, and the learned SE(3) forecast (orange), with SR of the corresponding policies as dots. **(e) Predict / context.** Passage tasks only. x: clearance (mm); y: SR; lines: no context flow, 14^2, 28^2, 56^2 grids, future latent; markers: each grid's measured e_F. **(f) Act / coupling.** x: added latency; y: ΔSR vs. training-only world supervision (R1); lines: hidden-state access (R2), bidirectional (R4), decoded geometry (R5), world losses off (R0, dotted); stationary targets as points at the right. 95% paired bootstrap bands, held-out scenes, three seeds; one color per family across panels.

Figure 4. **Measured trends against the predictions of the budget.** (a) The success of reactive policies collapses onto one curve of the uncompensated motion relative to the tolerance, and motion compensation shifts this curve. (b) The SE(3) history helps beyond the critical lead and more with latency, while visual histories stay flat. (c) Its value persists up to the predicted noise level $\sigma^\star$. (d) Learned SE(3) forecasts beat extrapolation on non-constant motion. (e) A coarse context flow suffices until the clearance reaches its error. (f) Training-only world supervision matches online use of the forecast at every latency. Bands show 95% paired bootstrap intervals.

training cost. As additional training targets, however, depth, DINO features, and moving-region RGB add [X.X] points.

Act. Indexing in physical time matters as predicted. Shifting the horizons by 50 ms lowers SR by [X.X] points on the fastest third of the scenes and by [X.X] points for stationary targets, and counting leads in control steps from the first command costs [X.X] points when the test latency differs from the training latency. Training-only world supervision with action-only deployment performs on par with the couplings that read the world ([±X.X] points at 0 ms and [±X.X] at 150 ms, Fig. 4f) at [X.X] times lower time-to-action. Disabling the world losses costs [X.X] points, and this gap grows with the lead and vanishes for stationary targets. The benefit therefore comes from training lead-conditioned actions jointly with forecasts of the scene at their application times.

5.4. Ablations and Pretraining

Removing the SE(3) history, the world losses, or the lead embedding lowers Dynamic-14 SR by [X.X], [X.X], and [X.X] points, and by [X.X], [X.X], and [X.X] under 150 ms of added delay (Table 4). On the real robot, the DROID base raises SR by [X.X] points over the generic initialization (Table 5). On Dynamic-14, post-training the base changes SR by [X.X] points with all demonstrations and by [X.X] with 10% of them, and a base pretrained with actions only

Table 3. **Controlled design comparisons** on held-out scenes (three training seeds). ⋆ marks the configuration selected on development scenes, and each block varies one axis. SR, SR_{150}: success (%) without and with 150 ms of added delay. e_t: contact-time error (ms). e_p: position error at first contact (mm). p95: time-to-action (ms).

Variant	SR	SR_{150}	e_t	e_p	p95
Remember (current frame, current pose, robot history fixed)					
Current frame only	—	—	—	—	—
+ RGB window (8 frames)	—	—	—	—	—
+ optical flow, motion statistics	—	—	—	—	—
+ video-VAE latent	—	—	—	—	—
+ SE(3) history, causal⋆	—	—	—	—	—
+ SE(3) history, oracle	—	—	—	—	—
Predict (future representation)					
No future	—	—	—	—	—
Target tracks + flow	—	—	—	—	—
Target tracks → SE(3) fit + flow	—	—	—	—	—
Target SE(3) only	—	—	—	—	—
SE(3) + 14 × 14 flow⋆	—	—	—	—	—
SE(3) + 28 × 28 flow	—	—	—	—	—
SE(3) + future RGB/latent	—	—	—	—	—
Act (world–action coupling)					
World as training target only⋆	—	—	—	—	—
Action reads world states	—	—	—	—	—
Bidirectional world states	—	—	—	—	—
Action reads decoded geometry	—	—	—	—	—

Table 4. **Ablations of the final model** on held-out Dynamic-14 scenes and DOMINO (SR, %). SR_{150} adds 150 ms of delay.

Model	SR	SR_{150}	DOMINO
SE(3)-WAM	—	—	—
Removed			
SE(3) history (current pose only)	—	—	—
world losses (R0)	—	—	—
lead embedding (latency unseen)	—	—	—
appearance targets (training only)	—	—	—
Replaced by			
leads in control steps	—	—	—
action reads world states	—	—	—
DROID-base initialization	—	—	—

recovers [XX]% of this low-data gain (Appendix E).

5.5. Real-Robot Experiments

A seven-DoF Franka arm with a parallel-jaw gripper must grasp an object that a belt conveyor carries past it, lift it, and hold it for 0.5 s, observed by a calibrated RGB-D camera (Fig. S4). The three objects differ in how much their orientation matters: a box with unequal sides, a slender can, and a cup with one handle. The belt runs at 0.04, 0.08, or 0.12 m/s. Each method is fine-tuned on the same [300] teleoperated demonstrations with three seeds and runs 180 trials, 20 per object and speed, in three blocks of shared trial keys with one seed per block. SE(3)-WAM succeeds in [XX.X]% of its trials (95% interval [[a, b]]), against [XX.X]% for $\pi_{0.5}$ and [XX.X]% without the DROID base (Table 5). Its advantage over $\pi_{0.5}$ grows from [X.X] points at 0.04 m/s to [X.X] at 0.12 m/s and is largest on [the cup]. Its remaining failures are mostly [late closures after tracking losses under the gripper].

Table 5. **Real-robot conveyor grasp.** Success rate (%) by object and by belt speed (slow, mid, fast: 0.04, 0.08, 0.12 m/s), 60 trials each, and over all 180 trials of a method. Every method is fine-tuned with three seeds, one per block of 60 trials. Binomial 95% intervals are given in Appendix F.3.

	Object			Belt speed			
Method	Box	Can	Cup	Slow	Mid	Fast	All
$\pi_{0.5}$ [31]	—	—	—	—	—	—	—
SE(3)-WAM, no DROID base	—	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—	—

6. Conclusion

We asked what a policy needs to know, and for which instant, to act on moving objects. An error budget in physical time points to the target's metric motion, its SE(3) trajectory with a coarse flow of the context, and commands indexed by their application time. SE(3)-WAM follows these choices and, in real time, improves on prior policies in simulation and on a real conveyor. The paused-physics comparison shows where the gain comes from. Large video models are strong policies when time stands still, and they lose this edge once the scene keeps moving while they compute. For world–action models, our results favor supervising the future at the instants when commands apply and keeping the deployed policy fast, over generating the future at test time.

Limitations. The budget assumes a rigid target that moves on its own until contact, so deformable or articulated targets and post-contact dynamics are outside its scope. SE(3)-WAM depends on a pose tracker whose failures propagate to the policy. Poses from a monocular 3D detector remove the need for an object model but cost accuracy, and the regressed target forecast cannot represent multimodal futures, e.g., before an impact. Lifting DROID to 4D labels costs substantial computation. Finally, our tasks have a single target and slow context motion, and the real-robot study covers one task on one platform.

LLM Usage Statement

[To be completed before submission: actual tools used, affected parts of the research and manuscript, and extent of use.]

References

[1] Iretiayo Akinola, Jingxi Xu, Shuran Song, and Peter K. Allen. Dynamic grasping with reachability and motion awareness. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 9422–9429, 2021. 1, 2

[2] Peter K. Allen, Aleksandar Timcenko, Billibon Yoshimi, and Paul Michelman. Automated tracking and grasping of a moving object with a robotic hand-eye system. *IEEE Transactions on Robotics and Automation*, 9(2):152–165, 1993. 2

[3] Ronald Azuma and Gary Bishop. Improving static and dynamic registration in an optical see-through HMD. In *Proceedings of SIGGRAPH*, pages 197–204, 1994. 2

[4] Yaakov Bar-Shalom, X. Rong Li, and Thiagalingam Kirubarajan. *Estimation with Applications to Tracking and Navigation*. Wiley, 2001. 2, 3, 6, 13, 28

[5] Berthold Bäuml, Thomas Wimböck, and Gerd Hirzinger. Kinematically optimal catching a flying ball with a hand-arm-system. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2010. 2

[6] Kevin Black, Manuel Y. Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. In *Advances in Neural Information Processing Systems*, 2025. 1, 2

[7] Arunkumar Byravan and Dieter Fox. SE3-Nets: Learning rigid body motion using deep neural networks. In *IEEE International Conference on Robotics and Automation*, 2017. 2

[8] Arunkumar Byravan, Felix Leeb, Franziska Meier, and Dieter Fox. SE3-Pose-Nets: Structured deep dynamics models for visuomotor control. In *IEEE International Conference on Robotics and Automation*, 2018. 2

[9] Chongkei Chang and Zhidong Deng. DynaWM: A base-VLA-guided world foundation model for moving-object manipulation. *arXiv preprint arXiv:2607.02604*, 2026. 2

[10] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin C. M. Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 1, 6, 7, 29

[11] David Emukpere, Romain Deffayet, Bingbing Wu, Romain Brégier, Michael Niemaz, Jean-Luc Meunier, Denys Proux, Jean-Michel Renders, and Seungsu Kim. Disentangled object-centric image representation for robotic manipulation. *arXiv preprint arXiv:2503.11565*, 2025. 2

[12] Heng Fang, Shangru Li, Shuhan Wang, Xuanyang Xi, Dingkang Liang, and Xiang Bai. Towards generalizable robotic manipulation in dynamic environments. *arXiv preprint arXiv:2603.15620*, 2026. 1, 2, 6, 7, 28, 29

[13] Gunnar Farnebäck. Two-frame motion estimation based on polynomial expansion. In *Proceedings of the Scandinavian Conference on Image Analysis*, pages 363–370, 2003. 20

[14] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022. 4, 17, 20

[15] B. Hove and Jean-Jacques E. Slotine. Experiments in robotic catching. In *American Control Conference*, 1991. 2

[16] Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, Jianke Zhang, Koushil Sreenath, Chaochao Lu, and Jianyu Chen. Video prediction policy: A generalist robot policy with predictive visual representations. In *International Conference on Machine Learning*, 2025. 2

[17] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. PointWorld: Scaling 3D world models for in-the-wild robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. 2, 5, 25

[18] Weikai Huang, Jieyu Zhang, Sijun Li, Taoyang Jia, Jiafei Duan, et al. WildDet3D: Scaling promptable 3D detection in the wild. *arXiv preprint arXiv:2604.08626*, 2026. 2, 4, 18, 26

[19] Seth Hutchinson, Gregory D. Hager, and Peter I. Corke. A tutorial on visual servo control. *IEEE Transactions on Robotics and Automation*, 12(5):651–670, 1996. 2

[20] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024. 2, 5, 25

[21] Shuang Li, Yihuai Gao, Dorsa Sadigh, and Shuran Song. Unified video action model. In *Proceedings of Robotics: Science and Systems*, 2025. 2

[22] Haoran Liao, Pengyue Wang, Shuoyu Chen, Kehan Cheng, Xuhang Chen, Yuhao Lin, Mu Lin, Zhizhao Liang, Xiaoyi Fan, Chengyi Xing, Dan Niu, Yi-Lin Wei, and Wei-Shi Zheng. DynamicManip: Enabling dynamic manipulation from a single static demonstration. *arXiv preprint arXiv:2608.01452*, 2026. 2

[23] Minghui Lin, Pengxiang Ding, Shu Wang, Zifeng Zhuang, Yang Liu, Xinyang Tong, Wenxuan Song, Shangke Lyu, Siteng Huang, and Donglin Wang. HiF-VLA: Hindsight, insight and foresight through motion representation for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. 1

[24] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. 4

[25] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2023. 2, 6, 29

[26] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *European Conference on Computer Vision*, 2024. 5, 18, 26

[27] Yushan Liu, Jingjing Fan, Shoujie Li, Yifan Xie, Xiao-Ping Zhang, and Wenbo Ding. PACT-WAM: Predicting actions and visual foresight with compact temporal encoding for robot manipulation. *arXiv preprint arXiv:2602.15882*, 2026. 1, 2, 20

[28] Yushan Liu, Peibo Sun, Shoujie Li, Yifan Xie, Lingfeng Zhang, Xintao Chao, Shiyuan Dong, Fang Chen, Xiao-Ping Zhang, and Wenbo Ding. OA-WAM: Object-addressable world action model for robust robot manipulation. *arXiv preprint arXiv:2605.06481*, 2026. 2

[29] Yunfan Lou, Hewen Gao, Xiyu Zhu, Zhuoran Qiao, Xuan Han, Yifan Yang, Yifan Ye, Boxian Yao, and Zhibo Pang. DynamicWAM: Dual-path motion conditioning for world-action models in dynamic manipulation. *arXiv preprint arXiv:2608.00793*, 2026. 1, 2, 6, 7, 20, 29

[30] Henrique Morimitsu, Xiaobin Zhu, Roberto M. Cesar, Xiangyang Ji, and Xu-Cheng Yin. DPFlow: Adaptive optical flow estimation with a dual-pyramid framework. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17810–17820, 2025. 26

[31] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. 1, 6, 7, 8, 29

[32] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. In *International Conference on Learning Representations*, 2025. 2, 4, 18, 26

[33] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025. 4, 17, 18, 20, 22

[34] Otto J. M. Smith. Closer control of loops with dead time. *Chemical Engineering Progress*, 53(5):217–219, 1957. 2

[35] Shahram Najam Syed, Arthur Jakobsson, Haoran Hao, and Jeffrey Ichnowski. Intercepting the future: Latent-space predictive world model for dynamic VLA manipulation. *arXiv preprint arXiv:2606.02486*, 2026. 1, 2

[36] Jiaming Tang, Yufei Sun, Yilong Zhao, Shang Yang, Yujun Lin, Zhuoyang Zhang, James Hou, Yao Lu, Zhijian Liu, and Song Han. VLASH: Real-time VLAs via future-state-aware asynchronous inference. *arXiv preprint arXiv:2512.01031*, 2025. 1, 2, 4

[37] Zachary Teed and Jia Deng. RAFT-3D: Scene flow using rigid-motion embeddings. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021. 2

[38] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. 5, 13

[39] Changhao Wang, Jaden Clark, and Shuran Song. Chronos policy: Learning robot policies on physical time. *arXiv preprint arXiv:2610.13929*, 2026. 1, 2

[40] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. VGGT: Visual geometry grounded transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025. 2

[41] Ruicheng Wang, Sicheng Xu, Yue Dong, Yu Deng, Jianfeng Xiang, Zelong Lv, Guangzhong Sun, Xin Tong, and Jiaolong Yang. MoGe-2: Accurate monocular geometry with metric scale and sharp details. *arXiv preprint arXiv:2507.02546*, 2025. 26

[42] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. OpenWAM: An open, modular exploration towards systematic world-action model pretraining. *arXiv preprint arXiv:2609.07398*, 2026. 1, 2, 6, 7, 29

[43] Bowen Wen, Wei Yang, Jan Kautz, and Stan Birchfield. FoundationPose: Unified 6D pose estimation and tracking of novel objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17868–17879, 2024. 4, 18

[44] Chuan Wen, Xingyu Lin, John So, Kai Chen, Qi Dou, Yang Gao, and Pieter Abbeel. Any-point trajectory modeling for policy learning. In *Proceedings of Robotics: Science and Systems*, 2024. 2

[45] Yuxi Xiao, Qianqian Wang, Shangzhan Zhang, Nan Xue, Sida Peng, Yujun Shen, and Xiaowei Zhou. SpatialTracker: Tracking any 2D pixels in 3D space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. 2

[46] Haozhe Xie, Beichen Wen, Jiarui Zheng, Zhaoxi Chen, Fangzhou Hong, Haiwen Diao, and Ziwei Liu. DynamicVLA: A vision-language-action model for dynamic object manipulation. *arXiv preprint arXiv:2601.22153*, 2026. 1, 2, 6, 7, 29

[47] Mengda Xu, Zhenjia Xu, Yinghao Xu, Cheng Chi, Gordon Wetzstein, Manuela Veloso, and Shuran Song. Flow as the cross-domain manipulation interface. In *Conference on Robot Learning*, 2024. 2

[48] Zhenjia Xu, Zhanpeng He, Jiajun Wu, and Shuran Song. Learning 3D dynamic scene representations for robot manipulation. In *Conference on Robot Learning*, 2020. 2

[49] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026. 1, 2

[50] Shanglin Yuan, Weiheng Zhao, Xin Shi, Haoyi Jiang, Xianda Guo, Liu Liu, Wenyu Liu, Wei Sui, and Xinggang Wang. DreamWAM: Beyond RGB future prediction for world action models. *arXiv preprint arXiv:2608.04996*, 2026. 2

[51] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026. 2

[52] Peng Yun, Shouwang Huang, Hao Li, Jinxi Li, Jianan Wang, and Bo Yang. PhysMani: Physics-principled 3D world model for dynamic object manipulation. *arXiv preprint arXiv:2607.01938*, 2026. 2

[53] Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations. In *Proceedings of Robotics: Science and Systems*, 2024. 2

[54] Chuhan Zhang, Guillaume Le Moing, Skanda Koppula, Ignacio Rocco, Liliane Momeni, Junyu Xie, Shuyang Sun, Rahul Sukthankar, Joëlle K. Barral, Raia Hadsell, Zoubin Ghahramani, Andrew Zisserman, Junlin Zhang, and Mehdi S. M. Sajjadi. Efficiently reconstructing dynamic scenes one D4RT at a time. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. 2

[55] Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, XinQiang Yu, Jiazhao Zhang, Runpei Dong, Jiawei He, He Wang, Zhizheng Zhang, Li Yi, Wenjun Zeng, and Xin Jin. DreamVLA: A vision-language-action model dreamed with comprehensive world knowledge. In *Advances in Neural Information Processing Systems*, pages 24195–24228, 2025. 1, 2, 6, 7, 29

[56] Yifan Zhang, Ruiping Wang, and Xilin Chen. Dynamic behavior cloning with temporal feature prediction: Enhancing robotic arm manipulation in moving object tasks. *IEEE Robotics and Automation Letters*, 10(6):5209–5216, 2025. 2

[57] Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, Ankur Handa, Tsung-Yi Lin, Gordon Wetzstein, Ming-Yu Liu, and Donglai Xiang. CoT-VLA: Visual chain-of-thought reasoning for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025. 2

[58] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023. 6, 7, 29

[59] Ruijie Zheng, Yongyuan Liang, Shuaiyi Huang, Jianfeng Gao, Hal Daumé III, Andrey Kolobov, Furong Huang, and Jianwei Yang. TraceVLA: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. In *International Conference on Learning Representations*, 2025. 1

[60] Hongyan Zhi, Peihao Chen, Siyuan Zhou, Yubo Dong, Quanxi Wu, Lei Han, and Mingkui Tan. 3DFlowAction: Learning cross-embodiment manipulation from 3D flow world model. *arXiv preprint arXiv:2506.06199*, 2025. 2

[61] Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. In *Proceedings of Robotics: Science and Systems*, 2025. 2

[62] Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Kevin Lin, Abhiram Maddukuri, Soroush Nasiriany, and Yifeng Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020. 5, 13

Supplementary Material

This supplement gives the derivation of the error budget (Appendix A), the benchmark (Appendix B), implementation details (Appendix C), the design study (Appendix D), the DROID pretraining (Appendix E), the baselines, external benchmarks, real robot, and timing (Appendix F), and additional results (Appendix G). Colored dashes and bracketed entries denote values pending the final runs.

A. The Interaction Error Budget

This appendix derives the budget of Sec. 3, extends it to rotation and chunked execution, works out its predictions, gives magnitudes for Dynamic-14, and states where it does not apply.

A.1. Translation

Let the true target position at the application time be $\mathbf{p}(\tau^{\mathrm{a}}) = \mathbf{p}(\tau_t) + \mathbf{v}\Delta + \boldsymbol{\rho}$ and the policy's estimate $\hat{\mathbf{p}}(\tau^{\mathrm{a}}) = \hat{\mathbf{p}}(\tau_t) + \hat{\mathbf{v}}\hat{\Delta} + \hat{\boldsymbol{\rho}}$. Subtracting and adding $\mathbf{v}\hat{\Delta}$ gives the identity

$$\begin{aligned}\hat{\mathbf{p}}(\tau^{\mathrm{a}}) - \mathbf{p}(\tau^{\mathrm{a}}) = \big[\hat{\mathbf{p}}(\tau_t) - \mathbf{p}(\tau_t)\big] + (\hat{\mathbf{v}} - \mathbf{v})\hat{\Delta} \\ + \mathbf{v}(\hat{\Delta} - \Delta) + (\hat{\boldsymbol{\rho}} - \boldsymbol{\rho}),\end{aligned} \tag{S1}$$

and the triangle inequality yields Eq. 4. Since $\mathbf{v}$ is the velocity at τ_t, $\boldsymbol{\rho}$ is exactly the part of the motion that constant-velocity extrapolation misses. Under a constant acceleration $\mathbf{a}$, $\boldsymbol{\rho} = \frac{1}{2}\mathbf{a}\Delta^2$, and an acceleration estimate from the history removes part of e_{dyn}. Impacts, rolling contact, and friction changes leave a remainder that only a model of the dynamics can reduce. For a learned predictor, the split of its estimate into $\hat{\mathbf{v}}\hat{\Delta}$ and $\hat{\boldsymbol{\rho}}$ is not identified, because any predicted position can be written in this form. We therefore use the budget to say which information each term requires. Our tests do not separate the terms inside a learned predictor. Instead, they vary the lead, the speed, the pose noise, and the timing, which act on known terms, and they compare variants that differ in the information they receive (Appendix A.5).

A.2. Rotation and the Contact Point

We write $R(\tau^{\mathrm{a}}) = \mathrm{Exp}(\boldsymbol{\omega}\Delta + \boldsymbol{\eta})R(\tau_t)$ and $\hat{R}(\tau^{\mathrm{a}}) = \mathrm{Exp}(\hat{\boldsymbol{\omega}}\hat{\Delta} + \hat{\boldsymbol{\eta}})\hat{R}(\tau_t)$. When R and $\hat{R}$ rotate about a common fixed axis, the exponentials compose additively, and

$$\theta = \big\| \operatorname{Log}(\hat{R}(\tau^{\mathrm{a}})R(\tau^{\mathrm{a}})^{\top})^{\vee} \big\| \le \theta_{\mathrm{loc}} + \hat{\Delta}\, e_{\omega} + \|\boldsymbol{\omega}\|\, e_{\tau} + e_{\eta}, \tag{S2}$$

where θ_{loc} is the current orientation error and e_{ω} and e_{η} are defined as for translation. For general rotations, the Baker–Campbell–Hausdorff formula adds terms of second order in the angles. A contact point at $\mathbf{c}$ in the object frame, with $\|\mathbf{c}\| = r$, has the world position $\mathbf{p} + R\mathbf{c}$, so its error is $(\hat{\mathbf{p}} - \mathbf{p}) + (\hat{R}R^{\top} - I)R\mathbf{c}$. The operator norm of $\hat{R}R^{\top} - I$ for a rotation by θ is $2\sin(\theta/2)$, which gives

$$e_{\mathrm{c}} \le \|\hat{\mathbf{p}} - \mathbf{p}\| + 2r\sin(\theta/2) \le \|\hat{\mathbf{p}} - \mathbf{p}\| + r\theta, \tag{S3}$$

that is, Eq. 3. Orientation errors therefore enter the contact geometry scaled by the lever arm, and a grasp or placement that depends on orientation needs an SE(3) target state. For rotationally symmetric targets, rotation about the symmetry axis leaves every contact unchanged, and we exclude it from inputs, labels, and errors.

A.3. Context and Clearance

Let $\mathcal{P}$ be the set of context points and times that the executed path passes, and suppose that the path keeps every point $\mathbf{x}$ of the robot and the carried object at least m away from the *predicted* context positions $\hat{\mathbf{y}}_t(\mathbf{z}, \tau)$ for $(\mathbf{z}, \tau) \in \mathcal{P}$. A collision requires $\mathbf{x} = \mathbf{y}_t(\mathbf{z}, \tau)$ for some $(\mathbf{z}, \tau) \in \mathcal{P}$, and then $\|\hat{\mathbf{y}}_t(\mathbf{z}, \tau) - \mathbf{y}_t(\mathbf{z}, \tau)\| = \|\hat{\mathbf{y}}_t(\mathbf{z}, \tau) - \mathbf{x}\| \ge m$, which is Eq. 5. Context precision finer than m thus has no first-order effect on success. The bound concerns the size of the error and says nothing about sampling. A flow sampled on a grid misses structures thinner than its spacing and errs at motion boundaries. Both effects enter the path error e_F, which we measure on held-out points along the executed paths (Appendix D.3) instead of inferring it from the grid spacing. The margin is itself a design variable, since a policy can keep a larger margin when its context estimate is coarse, at some cost in reachability or time.

A.4. Leads Under Chunked Execution

With chunks of H commands, of which the first k are executed before the next chunk replaces the remainder, the relevant leads are $\Delta_{t,j} = \ell_t + j\Delta_a$ for $j < k$. The forecast horizons must therefore cover $[\ell_t, \ell_t + (k-1)\Delta_a]$, and we interpolate states between horizons (linearly in translation and along the geodesic in rotation). With latencies up to 250 ms and replanning every 100 ms, the leads stay within 350 ms, inside the 400 ms of the forecast. Because horizons and leads are indexed by physical time, a change in latency shifts which horizons are read, and no retraining is needed. The ablation "leads in control steps" (Table 4) instead counts leads from the first command in control periods, as policies do that treat observation time and execution time as equal. It is trained at the training latency and refers to the wrong instants when the latency changes.

A.5. Predictions

Critical lead. A policy that does not compensate motion ($\hat{\mathbf{v}} = 0$, $\hat{\boldsymbol{\rho}} = 0$) incurs $\|\mathbf{v}\|\Delta$ in addition to e_{loc} and $\|\boldsymbol{\rho}\|$. With a small localization error and smooth motion, its contacts fail once $\|\mathbf{v}\|\Delta > \varepsilon$, i.e., beyond the critical lead $\ell^{\star} = \varepsilon/\|\mathbf{v}\|$, and its success depends on the task, the speed, and the latency only through $\|\mathbf{v}\|\Delta/\varepsilon$. Table S3 lists ε, $\|\mathbf{v}\|$, and $\ell^{\star}$ for every task. Fig. 4a plots the success of each method against $\|\mathbf{v}\|\Delta/\varepsilon$, computed per episode at the first command's application time and pooled over tasks, speeds, and latencies, together with the Spearman correlation and a

fitted logistic curve for the reactive policies.

Pose noise. A least-squares velocity estimate from n equally spaced poses that span T, each with independent noise σ per axis, has the standard deviation $\sigma\sqrt{12(n-1)/(n(n+1))}/T \approx \sigma\sqrt{12/n}/T$ [4]. A pose history therefore keeps $\hat{\Delta}e_v$ below the tolerance up to $\sigma^\star = \varepsilon T\sqrt{n}/(\Delta\sqrt{12})$. For $\varepsilon = 10\,\text{mm}$, $T = 350\,\text{ms}$, $n = 8$, and $\Delta = 200\,\text{ms}$, this gives $\sigma^\star \approx 14\,\text{mm}$. A learned estimator with a motion prior can do better, so $\sigma^\star$ is the prediction for a model-free linear estimate, and Fig. 4c compares it with the measured crossover.

Horizon shift. Reading the forecast at $\tau^{\text{a}} + \delta$ instead of τ^{a} sets $e_\tau = |\delta|$ and adds $\|\mathbf{v}\|\,|\delta|$ to the contact error. The penalty of a fixed shift therefore grows with the target's speed and vanishes for stationary targets.

Clearance. A context forecast with path error e_F avoids collisions while $m > e_F$. A coarse flow should therefore match finer flows at clearances above its measured e_F and fall behind below it.

A.6. Magnitudes in Dynamic-14

Table S1 evaluates the terms for representative Dynamic-14 settings (Table S4). Each of them is comparable to or larger than the millimeter-scale tolerances of the success conditions (Table S2).

Table S1. Illustrative magnitudes of the budget's terms for a 200 ms lead. The values follow from the stated quantities and are not measurements.

Term	Setting	Size
$\|\mathbf{v}\|\Delta$	conveyor at 0.10 m/s	20 mm
$\hat{\Delta}e_v$	10% speed misestimate	2.8 mm
$\|\mathbf{v}\|e_\tau$	50 ms timing error (one period)	7.0 mm
$2r\sin(\theta/2)$	$\omega = 1.2$ rad/s unpredicted, $r = 3$ cm	7.2 mm
e_{dyn}	ball on a 10° ramp, no acceleration model	24 mm
ε	grasp of the conveyor part	[X] mm
grid spacing	14×14 flow at the workspace	[X] mm

The ramp entry uses the acceleration of a solid sphere that rolls without slipping, $a = \frac{5}{7}g\sin\alpha \approx 1.2\,\text{m/s}^2$ at $\alpha = 10°$, and $\frac{1}{2}a\Delta^2$ for $\Delta = 0.2\,\text{s}$.

A.7. Scope

The budget addresses the approach to a rigid target whose motion is exogenous until contact. It does not cover deformable or articulated targets, whose motion field on $\mathcal{O}$ is not determined by six numbers. It also does not cover the motion after contact, which depends on the action itself and calls for action-conditioned prediction, the case in which letting the world read the action could matter. Scenes in which the target's role changes over time (e.g., sequential multi-object tasks) are outside its scope as well. Perception quality enters only through e_{loc} and e_v, and we measure it separately (Appendix E.4). The bound is not a probability of success. It identifies which errors scale with which factors, and Sec. 5.3 tests those scalings and the numbers above.

B. Dynamic-14

B.1. Tasks, Roles, and Terms

We built Dynamic-14 on MuJoCo and robosuite [38, 62]. It contains fourteen tasks in five families (Fig. S1). Table S2 lists their success conditions and Table S4 their scenario ranges. We will release the tasks, the accepted scenarios, the success checks, and the demonstrations.

Roles and critical leads. Table S3 assigns the bodies of each task to the two roles of Sec. 3. The target is the body whose pose error enters a contact or containment tolerance ε, and the context consists of bodies that only need to be avoided with clearance m. A moving receptacle is therefore a target, since its pose decides containment. The wire of the passage tasks plays both roles. Its pose at the entry is a target, because the wand must be aligned within 30 mm there, and its geometry along the loop is context, because any contact fails. The table also gives each task's speeds and its critical lead $\ell^\star = \varepsilon/\|\mathbf{v}\|$ (Appendix A.5), and its last column lists the terms of the budget that each task exercises. Conveyor and translating targets set the speed and the lead. Rotating targets, SE(3) placement, and the wire make orientation enter the contact. Ramps, low friction, and strikers produce motion that extrapolation misses, the occluder, wire, tray, and striker set the clearance, and the occluder also removes history.

We check success at every 2 ms physics step, after the ordered requirements of Table S2, and an interrupted continuous condition resets its counter. Terminal events fall into five categories that we report separately. An *invalid* event is a corrupted simulator state. *Boundary* events are a workspace or downstream exit, a drop, or an expired goal window. *Safety* events are unsafe contact, an excessive rim impulse, wire contact, a release impulse, early robot contact, unsupported tray contact, or a striker trap. *Task violations* are a release outside the goal, a path deviation or backtracking, a premature release, a deflection without capture, tipping, a missing or extra impact, or an insufficient change of outgoing motion. The last category is the *timeout*. Each family evaluates its events in a fixed order per physics step and keeps the first terminal event. The task-balanced score weights every task equally, including tasks on which some methods never succeed.

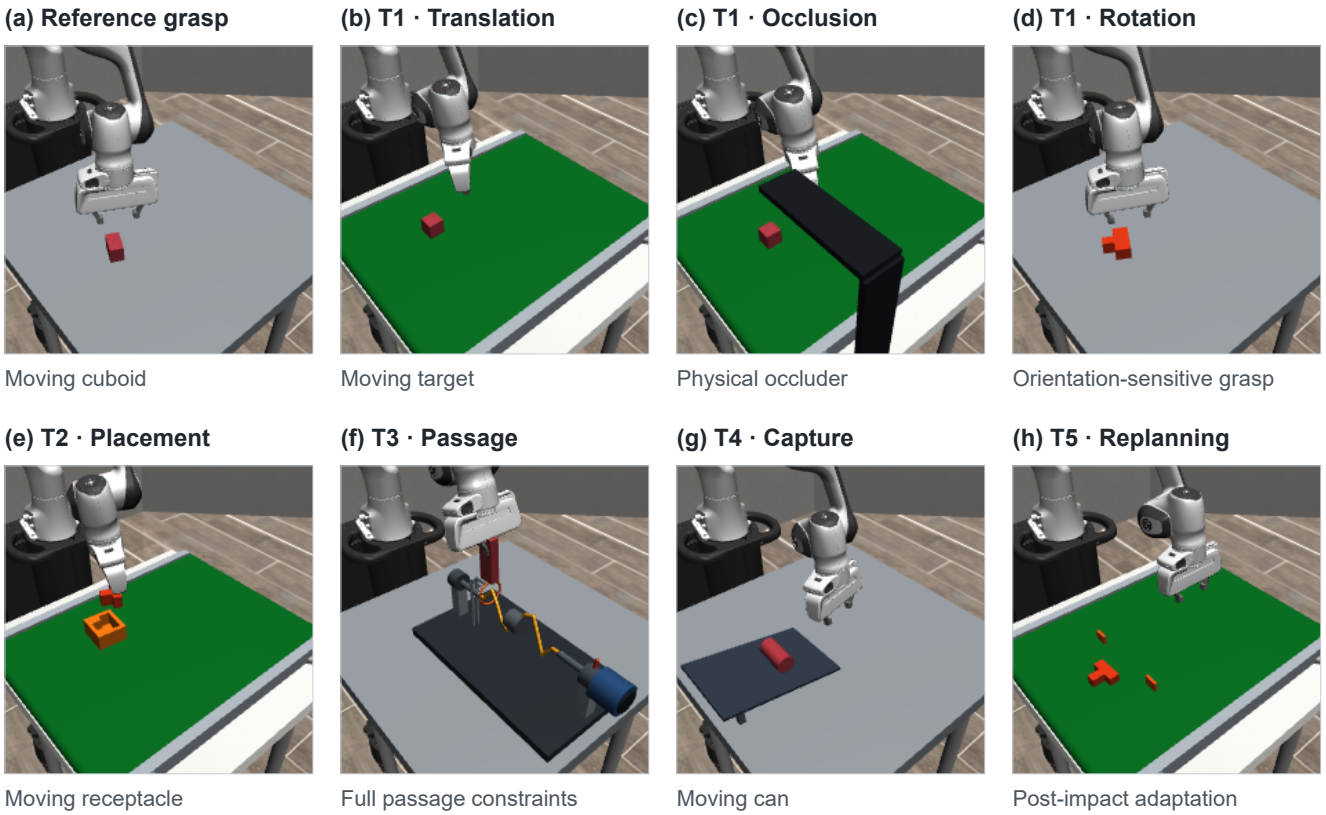


Figure S1. Dynamic-14 observations. Eight examples cover reference grasping, grasping a translating, occluded, or rotating target, placement into a moving receptacle, passage along a rotating wire, capture of a rolling object, and grasping after an impact. Each panel is the final observed, uncropped 224×224 agent-view RGB of one development scene. Table S2 lists all fourteen tasks.

B.2. Scenarios and Demonstrations

We sample continuous ranges uniformly and discrete choices with equal probability, and then apply physical-feasibility checks. Evaluation replays the stored accepted scenarios. Harmonic translation follows $\mathbf{p}(t) = \mathbf{c} + (0, v_y, 0)t + (A_x \sin(2\pi f_x t + \phi), 0, 0)$ with $\phi \sim \mathcal{U}[-\pi, \pi]$. Capture targets start at rest and move under gravity and friction on ramps of $[5, 15]^\circ$. We accept an occluded scene only if the target is visible for at least eight initial frames, fully occluded for 0.5–3.0 s, and visible again for at least three frames.

The training pool contains 11,200 demonstrations, 800 per task. They were generated by scripted experts with privileged simulator state, whereas the learned policies receive only the inputs of Sec. 4.1. We record the demonstrations at 20 Hz with camera images, depth, robot states, issued commands, and simulator ground truth for target poses and scene geometry. From these we compute the future labels, their validity masks, and the latency-shifted command labels. In simulation, the tracker's object model is the simulator's mesh of the target, the counterpart of a CAD model on a real robot.

B.3. Protocol and Sweeps

Executor. Every method runs under the same asynchronous executor. A chunk is requested every 100 ms of simulated time, or when the previous inference ends if that is later, and each command is executed at its application time. A new chunk replaces the unexecuted remainder of the previous one, and commands whose application time has passed by the time they become available are dropped and counted. Methods without a notion of application time receive their chunks as if issued at the end of inference, which is how they are deployed.

Real-time protocol. Unless noted otherwise, physics advances during inference by a latency drawn per query from the method's measured latency distribution on one RTX 5090 (Appendix F.4). The post-impact grasp additionally has a 50 ms command delay built into the task. Table S23 reports three alternatives. With *paused* physics, the simulation waits for inference, as in many simulated benchmarks. With *equal latency*, every method is charged the same latency, the p95 of SE(3)-WAM, regardless of its own speed. With *original executors*, each baseline keeps the execution scheme of its

Table S2. Success conditions of the fourteen Dynamic-14 tasks, with the thresholds of our task implementations. A task must pass its preceding stages and avoid terminal failures. All tasks run at 20 Hz with a 2 ms physics step; durations below refer to physical time. COM denotes center of mass.

Task	Ordered success conditions	Limit
grasp; t1_belt, t1_xyz, t1_so3, t1_occ	Approach within 0.10 m; bilateral fingerpad contact for 0.05 s; collision-geometry bottom at least 0.05 m above support; retain contact and clearance for 0.50 s. T1 adds conveyor drive, translation, rotation, or a physical occluder. SO3 has no additional success-angle threshold; unsafe occluder contact terminates OCC.	160 steps 8 s
t2_trans	Approach within 0.10 m. Entire object collision geometry inside the receptacle: planar clearance and floor gap both ≥ -0.002 m, top no higher than the opening. Release requires an opening command < -0.5 and no fingerpad contact. Relative linear/angular speeds ≤ 0.08 m/s and 0.8 rad/s for 0.10 s, followed by a 0.50 s stable dwell.	200 10 s
t2_se3	Approach within 0.05 m and 0.12 rad. Full T-shaped containment: planar clearance ≥ 0.002 m, floor gap in $[-0.002, 0.002]$ m, tilt ≤ 0.05 rad, top below opening. After release, relative speeds ≤ 0.02 m/s and 0.04 rad/s for 0.10 s, then a 0.50 s dwell. The 0.12 rad gate applies to approach.	200 10 s
t3_phase	The episode starts with the wand already grasped. Phase alignment: distance ≤ 0.090 m and loop-direction error ≤ 0.20 rad. Enter within 0.030 m at progress ≤ 0.15; reach historical maximum progress ≥ 0.60, then ≥ 0.85 and endpoint distance ≤ 0.055 m.	420 21 s
t3_full	Acquire the wand with bilateral contact for 0.05 s, then complete the phase stages above. In a later sample, retain current progress ≥ 0.85 and move at least 0.045 m from both path and endpoint. Finally open (< -0.5) with no fingerpad contact. Releasing before exit fails.	640 32 s
t4_sphere	Approach within 0.10 m; first qualifying contact after ramp exit; any tray contact before success fails. Bilateral hold for nominal 25 ms (12 physics samples, 24 ms). Raise COM by 0.05 m from its height when the bilateral-hold criterion is first met, and retain contact and height for 0.50 s.	120 6 s
t4_sphere_tabletop; t4_slider	Approach within 0.10 m; contact after ramp exit, while supported, in the capture region, and moving at planar speed ≥ 0.08 m/s. Bilateral hold for 0.05 s, COM lift ≥ 0.05 m from the first attainment of that hold criterion, then contact and height for 0.50 s.	120 6 s
t4_can	Same supported-capture sequence. Before first fingerpad contact, a can-axis absolute dot product with vertical > 0.45 fails as unstable tipping. The configuration does not require a transverse closing axis.	120 6 s
t5_replan	Exactly one selected-side impact group completes before robot contact. At 30 ms after the physical impact-end timestamp, branch-signed lateral speed ≥ 0.035 m/s and yaw rate ≥ 0.35 rad/s. Approach within 0.10 m; bilateral hold with grasp-axis error $\leq 18^\circ$ for 0.05 s; COM lift ≥ 0.05 m from the first attainment of that hold criterion and stable hold for 0.50 s. Impact-group lag is 12 ms and maximum contact is 0.10 s.	120 6 s

paper.

Latency. An added delay of 0, 50, 100, 150, or 200 ms is inserted after inference on top of the protocol's latency, and physics advances during it.

Speed. All exogenous velocities of a scene (belt, receptacle, wire, and the rate of harmonic motion) are multiplied by $s \in \{0.5, 1, 1.5\}$. Scenes that become physically infeasible at some factor are excluded for every method, and we report the remaining counts.

Clearance and grid resolution. The clearance sweep uses the passage tasks, whose wire is a context body. The free clearance between the wand's loop and the wire takes the values [X, X, X, X, X] mm, which bracket the spacings of the 14^2, 28^2, and 56^2 flow grids at the workspace ([X], [X], and [X] mm), while the object and the motion stay unchanged. Placement tolerances concern target precision (ε) and are not part of this sweep.

Pose noise. We add Gaussian noise to the tracker's past pose estimates (Appendix D.2).

Occlusion and stationarity. Occlusions of 0.5, 1, 2, and 3 s and an unobstructed control isolate history gaps. Stationary-target variants freeze the exogenous motion of every task and serve as the static control of Sec. 5.3.

B.4. Splits and Statistics

We use a development pool of 1,400 scenes (100 per task) for every design decision. All Dynamic-14 numbers that

Table S3. **Roles, tolerances, and critical leads of the Dynamic-14 tasks.** The target is the body whose pose error enters a contact or containment tolerance ε. The context comprises bodies that must only be avoided, with clearance m. Speeds follow the sampling ranges of Table S4 (capture tasks: at the ramp exit), and the critical lead $\ell^\star = \varepsilon/\|\mathbf{v}\|$ is given for the median speed. Terms: the parts of the budget that a task exercises. v: moving target. $r\theta$: orientation enters the contact. e_{dyn}: non-constant motion (harmonic, gravity, friction, impact). m: a context body must be avoided. Occ.: history gaps. Delay: command delay built into the task.

Task	Code	Target	Context	ε (mm)	$\|\mathbf{v}\|$ (m/s)	$\ell^\star$ (ms)	m (mm)	Terms
Harmonic grasp	grasp	block	table	[X]	0.03–0.20	[X]	–	v, $r\theta$, e_{dyn}
Conveyor grasp	t1_belt	part	belt	[X]	0.04–0.10	[X]	–	v
Translating grasp	t1_xyz	part	table	[X]	0.04–0.15	[X]	–	v, e_{dyn}
Rotating grasp	t1_so3	part ($\omega \leq 1.2$ rad/s)	table	[X]	0.04–0.15	[X]	–	v, $r\theta$, e_{dyn}
Occluded grasp	t1_occ	part	occluder canopy	[X]	0.04–0.10	[X]	[X]	v, m, occ.
Moving receptacle	t2_trans	receptacle	table	[X]	0.03–0.06	[X]	–	v
Rotating receptacle	t2_se3	T-shaped receptacle	table	[X]	0.03–0.06	[X]	–	v, $r\theta$
Wire passage	t3_phase	wire loop (entry)	rotating wire	30	[X]	[X]	swept	v, $r\theta$, m
Wire passage, full	t3_full	wire loop (entry)	rotating wire	30	[X]	[X]	swept	v, $r\theta$, m
Ball capture	t4_sphere	ball	tray, ramp	[X]	[0.4–0.7]	[X]	[X]	v, e_{dyn}, m
Ball capture, table	t4_sphere_tabletop	ball	table, ramp	[X]	[0.4–0.7]	[X]	–	v, e_{dyn}
Slider capture	t4_slider	slider	table, ramp	[X]	[0.4–0.8]	[X]	–	v, e_{dyn}
Can capture	t4_can	can	table, ramp	[X]	[0.4–0.7]	[X]	–	v, $r\theta$, e_{dyn}
Post-impact grasp	t5_replan	part	striker	[X]	0.12–0.14	[X]	[X]	v, $r\theta$, e_{dyn}, m, delay

we report, in the main paper and in this supplement, use a separate held-out set of 1,400 scenes, generated with the same ranges and with new source seeds that appear in no training, validation, or development episode. The first 100 feasible scenes in increasing seed order form each task's set.

The final model, its ablations, and the variants of Table 3 use training seeds 0, 1, and 2. The screening variants of Appendix D and the baselines use seed 0. On development scenes, three training seeds of the same configuration differed by up to [7.4] points in SR. Most of this spread came from the wire passage, whose success ranged from [0] to [71]% across seeds, and we do not interpret single-seed differences of a few points. Three seeds are too few to resample, so we average them per scene and bootstrap the held-out scenes within each task, jointly across methods, for paired 95% intervals. We treat differences whose interval lies within ± 2 points as ties.

B.5. Metrics

The task-balanced success rate is

$$\mathrm{SR}(m) = \frac{1}{14}\sum_{k=1}^{14}\frac{1}{n_k}\sum_{i=1}^{n_k} y_{mki}, \tag{S4}$$

for binary outcomes y_{mki} of method m on scene i of task k. The *feasible contact window* of a scene is the set of times at which a contact that satisfies the task's first success stage is kinematically possible. We compute it from the ground-truth target trajectory and the robot's reachable workspace. The *contact-time error* is the distance from the first-contact time to this window, and it is zero inside the window. The *position error at first contact* is the distance of the end effector from the target's grasp frame at that time. We report both on episodes that make contact. Failures without contact stay in the SR denominator, and their contact error is missing rather than zero. *Collisions* count safety events. *Time-to-action* is the wall-clock time from the arrival of an observation to the availability of the first command (Appendix F.4), and the *action age* is the application time minus the capture time.

Table S4. Scenario proposal ranges. Length, linear velocity, angle, and angular velocity use m, m/s, rad, and rad/s unless marked otherwise. Positions are in world coordinates except the stated ramp-local offsets. Continuous intervals are uniform proposals; discrete choices are equiprobable before feasibility checks. Evaluation replays the stored accepted scenarios, whose distribution need not remain uniform. Harmonic centers are not initial positions.

Task	Initial-state proposal	Motion or event parameters
grasp	Harmonic center $c_x \in [-.08, .08]$, $c_y \in [-.26, -.19]$, $z = .822$; yaw $[-1.1, 1.1]$.	$v_y \in [.025, .125]$, $A_x \in [.015, .035]$, $f_x \in [.3, .7]$ Hz; $\omega_z \in [-1.2, 1.2]$.
t1_belt	Target $x \in [-.08, .08]$, $y \in [-.26, -.18]$, $z = .822$; identity orientation, zero initial twist.	Belt $v_y \in [.04, .10]$.
t1_occ	Same target proposal as Belt; canopy at $(.322, -.29, 1.20)$.	Same belt range; clearance parameter .20; physical occlusion.
t1_xyz	Harmonic center $c_x \in [-.025, .025]$, $c_y \in [-.24, -.20]$, $z = .818$; yaw $[-.35, .35]$.	$v_y \in [.04, .09]$, $A_x \in [0, .03]$, $f_x \in [.25, .65]$ Hz; $\omega_z = 0$.
t1_so3	Same as XYZ.	Same translation; $\omega_z \in [-1.2, -.4] \cup [.4, 1.2]$, equal sign probabilities.
t2_trans	Goal $x \in [-.13, -.08]$, $y \in [-.18, -.12]$, $z = .825$; fixed orientation.	Goal $v_y \in [.03, .06]$; no rotation.
t2_se3	Same goal x, y range, $z = .804$; yaw $[-.3, .3]$.	Goal $v_y \in [.03, .06]$; $\omega_z \in [-.25, -.08] \cup [.08, .25]$, equal sign probabilities.
t3_phase	Wand $x, y \in [-.03, .03]$, $z = .870$, yaw $[0, 2\pi]$; wire phase $[-.25, .35]$.	Wire center $(.10, -.02, .94)$; world-X rotation $\omega_x = -\{2.5, 3, \ldots, 7.5\}^\circ$/s (11 equal-probability values).
t3_full	Wand at $(-.05, -.02, 1.04)$ with identity orientation; same wire phase range.	Same wire rotation; cradle and release requirement enabled.
t4_sphere	Ramp-local offsets: lateral $[-.015, .015]$, longitudinal/normal $(-.145, .030)$; lip $(.06, -.02, 1.10)$.	Gravity-driven from rest; gate release at tick 0; slope $[5, 15]^\circ$; target/ramp sliding friction $[.7, .9]$; unsupported capture.
t4_sphere_tabletop	Lateral $[-.015, .015]$, longitudinal/normal $(-.124, .034)$; lip $(0, -.06, .808)$.	Same slope and release; target/ramp friction $[.7, .9]$, table friction .35.
t4_slider	Lateral $[-.015, .015]$, longitudinal/normal $(-.129, .024)$; supported ramp.	Same slope and release; target/ramp friction $[.025, .045]$, table friction $[.012, .018]$.
t4_can	Lateral $[-.010, .010]$, ramp-axis yaw $[-4, 4]^\circ$; longitudinal position follows gate geometry and oriented can extent.	Same slope and release; target/ramp friction $[.25, .45]$, table friction $[.09, .15]$.
t5_replan	$x \in [-.005, .005]$, $y = -.29$, $z = .812$; yaw $[-.08, .08]$; zero initial twist.	Belt $v_y \in [.12, .14]$; equal-probability left/right striker triggered at $y = -.13$; stroke .098, pulse .35 s, actuator scale $[1, 1.1]$, force cap 4 N; command delay 50 ms.

C. Implementation

C.1. Condition Tokens

Current image. A frozen MAE ViT-B/16 [14] encodes the 224×224 agent-view RGB into 196 patch tokens. We selected it over frozen DINOv3 encoders [33] and a CNN trained with the policy on development scenes (Appendix D.1). Each patch receives a view embedding, a linear embedding of its mean valid metric depth, and a linear embedding of its back-projected base-frame position, which we compute from fixed source pixels with the camera intrinsics and extrinsics. These positions also enter 3D rotary position embeddings for the image queries and keys and for the scene queries, which share the same source points. Target, robot, and action tokens receive no rotary embedding. During training only, a linear decoder on the condition tokens reconstructs the RGB inside the target's current box (weight 0.1 in $\mathcal{L}_{\text{aux}}$), which ties the patch features to the target without adding any input.

Target-motion history. The causal tracker (Appendix C.2) emits poses at capture times. At each query, we resample the emitted track at the offsets $-350, -300, \ldots, 0$ ms, interpolating linearly in translation and along the geodesic in rotation between emitted estimates. We do not extrapolate and do not interpolate across gaps longer than two camera periods, so an offset without estimates on both sides is marked invalid. Each of the 8 motion tokens is a 25-dimensional descriptor $(\mathbf{p}, \log R, \Delta\mathbf{p}, \log \Delta R, \Delta t, \mathbf{v}, \boldsymbol{\omega}, \mathbf{a}, \boldsymbol{\alpha})$ with dimensions $(3, 3, 3, 3, 1, 3, 3, 3, 3)$. Slots 0–6 describe the seven intervals

between consecutive offsets, and slot 7 holds the current absolute pose. Velocities divide interval displacements by the elapsed time, and accelerations difference adjacent velocities. We standardize each dimension with training statistics, zero the invalid dimensions, and append three validity flags (pose, interval, and acceleration) before a $28 \to 768 \to 768$ GELU MLP with a time embedding. When the tracker reports the target as lost, a learned embedding replaces the pose.

Robot state. 8 end-effector states (position, rotation vector, and measured gripper opening) at the same offsets form robot tokens. One further token holds the end-effector pose commanded for the first application time $\tau^{\mathrm{a}}_{t,0}$, which is the reference of the chunk's increments (Appendix C.4).

C.2. Pose Tracker

SAM 2.1 (Hiera-small) propagates the target mask causally from a box in the first frame [32]. In simulation, this box comes from the target's rendered mask, and on the real robot it comes from Grounding DINO for the object phrase of the instruction [26]. FoundationPose registers and tracks the object in the full-resolution RGB-D frame ([640×480]) with its released weights [43]. In simulation, the object model is the target's simulator mesh. On the real robot, it is a CAD model or a set of [16] masked reference views captured once per object. The tracker runs in its own thread, and each estimate carries its capture time, its availability time, and a validity flag. The flag is false after a registration failure, a depth dropout over the mask, or a jump of more than [X] mm between frames.

Poses from the monocular 3D detector (Sec. 5.2) need no object model. For every masked frame, WildDet3D [18] predicts a metric 3D box of the prompted target from the RGB frame, its tight 2D box, and the intrinsics. Box axes are ambiguous up to the symmetries of a box, so we map each box to one body frame per episode by choosing, among the 24 axis permutations and sign flips, the one closest to the previous frame's orientation, as for DROID (Appendix E.2). For rotationally symmetric targets, we project the rotation about the symmetry axis out of the estimates, labels, and errors. Table S5 reports the accuracy of both trackers and the effect of the input resolution.

C.3. World Stream

The world consists of one learned object query ($\mathcal{Q}$) and one scene query per grid point ($\mathcal{F}$), all of width 768. They are initialized from learned embeddings and positional encodings, attend to the condition and to each other, and are regressed deterministically, so that neither future labels nor noise enter them. The readouts embed the physical offset δ_h of each horizon, which lets us supervise the same queries at other offsets, as in pretraining (Appendix E.1).

Table S5. **Causal tracker accuracy.** Median and p95 error of the emitted poses against ground truth, fraction of frames reported as lost, and held-out Dynamic-14 SR of SE(3)-WAM with each tracker (⋆: used).

Tracker	mm	°	Lost	SR
FoundationPose, full res.⋆	—	—	—	—
FoundationPose, 224 × 224	—	—	—	—
Monocular 3D detector (no object model)	—	—	—	—
Simulator poses (oracle)	0/0	0/0	0	—
Real robot, CAD prior	—	—	—	–
Real robot, reference views	—	—	—	–
Real robot, 3D detector	—	—	—	–

Target. An MLP reads the trajectory $\{(\mathbf{d}_h, \boldsymbol{\phi}_h)\}_{h=1}^{8}$ out of the final state of the object query. With valid horizons $\mathcal{V}$,

$$\mathcal{L}_P = \frac{1}{|\mathcal{V}|} \sum_{h \in \mathcal{V}} \left[\|\hat{\mathbf{d}}_h - \mathbf{d}_h\|^2 + \lambda_R \, d_{\mathrm{SO}(3)}\big(\hat{R}_h, R_h\big)^2 \right], \quad \text{(S5)}$$

with $\lambda_R = 0.1$ and $\hat{R}_h = \mathrm{Exp}(\hat{\boldsymbol{\phi}}_h^{\wedge}) R(\tau_t)$. For symmetric targets, $d_{\mathrm{SO}(3)}$ ignores rotation about the symmetry axis. We use the left-relative rotation vector and the cumulative translation instead of a twist exponential, which would couple translation to rotation through its Jacobian. The forecast is a conditional mean, so it averages over multimodal futures (e.g., the outcome of an impact before it happens).

Scene flow. The flow is the cumulative 3D displacement of $N = 196$ source points over the 8 horizons, $F \in \mathbb{R}^{N \times 8 \times 3}$, in the base frame, together with a visibility $o \in \{0,1\}^{N \times 8}$. Each scene query carries embeddings of its point's source location and role. The role (target, robot, or context) comes from the tracker's mask and from the robot's rendered or kinematic mask. A linear readout gives each point's displacement and visibility logit at every horizon. We divide the displacements by one fixed scale fitted on the training labels and use

$$\mathcal{L}_F = \frac{1}{|\mathcal{V}_F|} \sum_{(i,h) \in \mathcal{V}_F} o_{ih} \big\| \hat{F}_{ih} - F_{ih} \big\|^2 + \mathrm{BCE}(\hat{o}, o), \quad \text{(S6)}$$

where $\mathcal{V}_F$ holds the valid labels, so that an occluded point supervises only its visibility.

Appearance targets. Patch decoders on the final states of the scene queries predict three targets at every horizon: the metric depth of their patch (squared error on valid depth), the features of a frozen DINOv3 ViT-L/16 teacher [33] on the 14×14 grid after channel-wise layer normalization (squared error), and the RGB of moving surfaces (i.e., surfaces that move by more than 1 mm, including the robot), with a squared error inside that mask. Each target enters

$\mathcal{L}_{\text{aux}}$ with weight 0.1. We skip these heads at deployment, and no token reads their outputs, so they only shape the states of the scene queries.

C.4. Action Stream

Each chunk has 8 seven-dimensional commands, each with a base-frame translation increment, a left-relative rotation vector, and a gripper command. Command j is an increment from the pose commanded for $\tau^{\text{a}}_{t,j-1}$. For $j = 0$, it is the pose that the previous chunk, which is still executing, commanded for $\tau^{\text{a}}_{t,0}$. The chunk is thus anchored to poses that the controller knows exactly, instead of the arm's measured pose at capture. Each action token adds a sinusoidal embedding of its lead $\Delta_{t,j} = \ell_t + j\Delta_a$. During training, we draw a latency $\ell \sim \mathcal{U}[0, 250\,ms]$ per window and take as the label of command j the demonstration's commanded increment at $\tau_t + \ell + j\Delta_a$, interpolated between control steps. Normalization uses the mean and the population standard deviation of valid, non-repeated issued commands in the training split. Flow matching uses a unit Gaussian in these coordinates with one u per chunk, and the action tokens receive u through adaptive layer normalization. At deployment, we reverse the normalization, decode the rotation vectors, and clip translation and rotation to 0.10 m and 0.5 rad per axis. The gripper follows the sign of its command.

C.5. Coupling

The attention mask acts on four token groups, the condition ($\mathcal{Z}$), the object query ($\mathcal{Q}$), the scene queries ($\mathcal{F}$), and the actions ($\mathcal{A}$). Condition tokens attend only to $\mathcal{Z}$, the world queries attend to $\mathcal{Z}$, $\mathcal{Q}$, and $\mathcal{F}$, and the action tokens attend to $\mathcal{Z}$ and $\mathcal{A}$, plus whatever their coupling adds (Appendix D.4). A blocked edge has the logit $-\infty$ at every layer and head, both in training and at deployment. A world query that the action does not attend to therefore cannot influence it, and no future label reaches the action through the condition. All couplings use the same modules, initialization, and losses and differ only in the mask, or in R5 in the geometry encoder. We do not detach any gradient between the streams. Unless the action-to-world edge is open, the world attends neither to the actions nor to the flow time, so its states are computed once per query and reused across the Euler steps in R1, R2, and R5. In R3 and R4, they are recomputed at every step. In the selected coupling R1, no edge from the world to the action is open. The world losses shape the shared parameters, and deployment runs only the action stream with 5 Euler steps. The backbone shares the attention projections across groups and uses separate feed-forward networks (ratio 4) for the world queries and the action tokens. World queries use layer normalization and action tokens adaptive layer normalization, for 47.3M trainable backbone parameters. The alternative parameterization of Appendix D.4 differs in the number of trainable parameters by 0.013%.

C.6. Training Recipe

Table S6 lists the recipe for training on Dynamic-14. Each demonstration contributes windows whose query times are [8] control steps apart, about [14] per demonstration, each with its own latency draw. A global batch of 64 with twice the learning rate trained about [2.5] times faster in a pilot but was less stable across seeds, so we kept 16. One training run takes about 7.7 hours on eight RTX PRO 6000 GPUs. The DROID base and the controlled comparisons use the recipe of Appendix E.5.

Table S6. Training hyperparameters on Dynamic-14.

Item	Value
Backbone	4 layers, width 768, 8 heads, 47.3M parameters
Encoder	frozen MAE ViT-B/16, 224×224 input
Optimizer	AdamW, $\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$, weight decay 0.05
Learning rate	2×10^{-4}, 1,000 warm-up updates, cosine decay to 0.1
Batch, updates	global batch 16, 4 epochs (40,104 updates, 641,664 windows)
Loss weights	$\lambda_P = \lambda_F = 1$, $\lambda_R = 0.1$, $\lambda_{\text{aux}} = 0.1$ per target
Latency draw	$\ell \sim \mathcal{U}[0, 250\,ms]$ per window
Precision	BF16 with TF32, gradient clipping at 1, no dropout
Sampling	5 Euler steps for the action
Inference	FP16, compiled, condition tokens cached
Hardware	eight RTX PRO 6000 GPUs, about 7.7 hours per run

C.7. Deployment

Each query encodes the newest observation, reuses the cached condition tokens within the query, and issues a chunk whose commands are scheduled at their application times (Eq. 1). Before the query, the executor sets ℓ_t to its latency budget, the p95 of the last 50 measured inference times plus the actuation delay of the platform. If inference finishes early, the commands wait for their times. If it finishes late, the commands whose times have passed are dropped and the rest execute. Appendix B.3 describes the executor shared by all methods, and Appendix F.4 the timing measurements.

D. The Complete Design Study

Every comparison in this appendix varies one axis, keeps the others at the selected configuration, and uses the recipe of Appendix C.6 with the same scenes and inference seeds for every variant. We screened all variants with one training seed and made every selection decision on the development pool. The tables report the held-out scenes. The encoder and coupling comparisons and the rows that also appear in Table 3 average three training seeds, and the other rows use

Table S7. **Observation encoder** on held-out scenes (three training seeds), with every other component at the selected configuration (⋆: used). Params: parameters of the encoder. p95: time-to-action on one RTX 5090.

Encoder	Params	Trained	SR	SR_{150}	p95
CNN	—	yes	—	—	—
DINOv3 ViT-S/16	22M	frozen	—	—	—
DINOv3 ViT-L/16	300M	frozen	—	—	—
MAE ViT-B/16⋆	86M	frozen	—	—	—

one. In total, the study took about [XXXX] GPU-hours of training and [XXXXXX] evaluation episodes.

Selection. A candidate had to keep the p95 time-to-action below 200 ms and its peak memory below 80% of one RTX 5090 at deployment precision. Because single seeds differ by several points (Appendix B.4), we replaced the reference only when a candidate gained more than 3 points on development scenes with seed 0 and kept a gain with seeds 1 and 2. Otherwise we kept the cheaper or the simpler variant.

D.1. Shared Components

The observation encoder and the training-only appearance targets are shared by all three studies, and we fixed them first. We compared a CNN trained with the policy with frozen DINOv3 ViT-S/16 and ViT-L/16 [33] and MAE ViT-B/16 [14] encoders on development scenes (Table S7). The frozen MAE encoder had the best mean of three seeds, [66.8]%, but its seeds ranged from [63.7] to [71.1]% and overlapped with the DINOv3 encoders. We chose it for this mean and because it costs less than ViT-L/16. DINOv3 ViT-L/16 features remain one of the future appearance targets, which we selected together with the world representation (Appendix D.3).

D.2. Remember

Conditions. Every condition receives the current image, the *current* target pose from the causal tracker, eight robot states, the selected future targets, and the selected coupling. The conditions differ only in what they keep from the past 350 ms, which holds e_{loc} fixed and isolates the history's contribution to e_v. The RGB window keeps seven past frames of 196 patches each, sampled every 50 ms, without compression. Optical flow is bidirectional Farnebäck flow [13] on 64×64 RGB over four intervals of 100, 100, 100, and 50 ms that span the same 350 ms window, with forward–backward validity, and "flow + motion statistics" appends the twelve physical statistics per interval of DynamicWAM [29]. Codec vectors come from actual MPEG-4 P-frame motion vectors and are encoded like the flows, and the video latent is the posterior mean of the frozen Wan2.2 VAE, read by 56 learned queries. The compressed windows read past frames with a shared bank of eight queries, either uniformly or with more queries for newer frames, following the allocation of PACT-WAM [27]. The RGB-D window adds the metric position of every past patch and tests whether the missing ingredient is metric geometry rather than motion. Among the SE(3) variants, the position-only history removes orientation, the detector history replaces the object model by per-frame 3D boxes from a monocular detector (Appendix C.2), and the oracle history uses simulator poses. A final control removes the current target pose from the RGB window to measure what the current pose alone contributes.

Frozen readouts. To separate the information that a history encoding *contains* from the information that the policy *uses*, we fit ridge readouts on frozen features at three sites, the output of the history encoder, the final object query, and the input of the action head at the first function evaluation. The labels are the target position 50, 200, and 350 ms in the past, the velocity over the last interval, and the future displacement at 400 ms, all in the base frame. Each task assigns 60, 20, and 20 distinct development episodes to training, validation, and testing, with five windows per episode. We choose the penalty from $\{0.1, 1, 10, 100\}$ on the validation error, and a shuffled-label control bounds chance performance. Conditional readouts first fit a baseline from the current image, geometry, robot state, and current pose and then predict its residual from the site features, which tests what the history adds beyond the present.

Perturbations and interventions. Offline perturbations repeat the current image in all past slots or reverse the past images while keeping the current frame, the poses, and the noise fixed, and we measure the change of the first issued action. On the final model (Table S9), *stationary history* replaces every past pose with the current pose before the kinematics are recomputed, which removes displacement, velocity, and acceleration. *Time reversal* reverses the past poses. A *matched-current swap* substitutes the history of the nearest different development episode with the same task, object, and camera and with current target and end-effector poses within 5 mm and 5°. The *direction-ambiguous* subset additionally requires opposite last-interval velocities of at least 0.01 m/s.

Pose noise. We add Gaussian noise with standard deviation $\sigma_p \in \{0, 1, 2, 5, 10, 20, 40\}$ mm in position and $\sigma_R = \sigma_p/(30\,\text{mm})$ rad in rotation, so that both produce the same displacement at a 3 cm lever arm, independently to every past pose estimate before the kinematics are computed. The current pose is unchanged, so the test isolates the history. We compute the predicted crossover $\sigma^\star$ (Appendix A.5) per task from Table S3 with $T = 350$ ms, $n = 8$, and the realized leads, and mark it in Fig. 4c together with the tracker's measured error (Table S5).

Table S8. **Remember: complete history comparison** on held-out scenes (one training seed, three for rows that also appear in Table 3). All conditions keep the current image, the current target pose (except the last row), robot history, the selected future representation, and the selected coupling. SR (%) under added latency. Readout: frozen-ridge error of the target position 350 ms in the past (mm, lower is better). Δa: change of the first issued translation when past inputs are repeated (mm). Tokens: added history tokens. p95: time-to-action (ms).

	SR at added latency (ms)								
History	0	50	100	150	200	Readout	Δa	Tokens	p95
Current frame only	—	—	—	—	—	—	0	0	—
RGB window	—	—	—	—	—	—	—	1,372	—
RGB-D window	—	—	—	—	—	—	—	1,372	—
Optical flow	—	—	—	—	—	—	—	60	—
Flow + motion statistics	—	—	—	—	—	—	—	60	—
Codec motion vectors	—	—	—	—	—	—	—	56	—
Video-VAE latent	—	—	—	—	—	—	—	56	—
Compressed window (age)	—	—	—	—	—	—	—	56	—
Compressed window (uniform)	—	—	—	—	—	—	—	56	—
SE(3) history, causal*	—	—	—	—	—	—	—	8	—
SE(3) history, position only	—	—	—	—	—	—	—	8	—
SE(3) history, detector boxes	—	—	—	—	—	—	—	8	—
SE(3) history, oracle	—	—	—	—	—	—	—	8	—
SE(3) history + RGB window	—	—	—	—	—	—	—	1,380	—
RGB window, no current pose	—	—	—	—	—	—	—	1,371	—

Table S9. **Frozen-policy interventions.** Each row changes one input at inference only. History interventions use the final model. Hidden-state interventions use coupling R2, and geometry interventions use R5 (the final model reads no forecast). SR (%) on held-out scenes. Δa: mean change of the first issued translation (mm).

Input	Intervention	SR	Δa
History	none	—	0
	stationary (current pose repeated)	—	—
	time-reversed	—	—
	matched-current swap	—	—
	direction-ambiguous swap	—	—
World states (R2)	none	—	0
	matched-current swap	—	—
	horizon times shifted ±50 ms	—	—
Decoded geometry (R5)	none	—	0
	zero motion	—	—
	matched-current shuffle	—	—
	time shift ±50 ms	—	—
	oracle (pre-contact)	—	—

Occlusion and sampling. Occlusions of 0.5, 1, 2, and 3 s and an unobstructed control test history gaps. The sampling tests keep the 350 ms span and select 2, 4, or 8 offsets, with time embeddings at the actual offsets.

D.3. Predict

Representations. Table S10 defines the families of future targets. All targets are cumulative from the current state and cover the same eight horizons from 50 to 400 ms. Every row of Table S11 keeps the history, coupling, action loss, training budget, and world-stream capacity of the selected configuration and replaces only the world targets, which the scene queries regress through the interface of Appendix C.3. We therefore compare image-space targets at matched capacity. The frozen-video row instead reads latents that a pretrained image-to-video model predicts from the current frame, a future that does not depend on our capacity. The Kabsch row predicts the object tracks and fits SE(3) to them, so it receives the same information as the track row plus a rigidity prior. The last block keeps the selected world and varies only the training-only appearance targets, whose heads we drop at deployment, so it measures what these targets teach the scene queries.

On development scenes, most representations stayed within a few points of each other overall, and their differences concentrated in a few tasks. Target point tracks with the flow even reached a slightly higher overall SR than the SE(3) trajectory ([+1.7] points with one seed). We kept the SE(3) trajectory, which gives the pose directly and which the budget favors for rotating targets, and Table S11 compares both on the held-out scenes. Dense 2D displacement was the only future that clearly hurt ([−3.6] points against no future).

Table S10. Future-target families compared in the *Predict* study. Shape excludes batch and query axes. Displacements are cumulative from the current state, except for codec vectors, which keep the correspondence of the decoder. All targets come from the recorded demonstrations except the frozen video model, whose latents are its own predictions. Future depth, DINO, and dynamic RGB are also the training-only auxiliary targets of the selected model.

Target	Shape	Support, units, and validity
Target pose	8×6	Cumulative translation (m) and left-relative rotation vector (rad). Requires a valid pose and query.
Scene flow, $k \times k$ grid	$k^2 \times 8 \times 3$	Fixed grid of $k \in \{14, 28, 56\}$ points per side, in base-frame meters. Requires valid source depth and a finite future displacement.
Scene flow, object-sampled	$196 \times 8 \times 3$	Points sampled on the context masks by farthest-point sampling instead of a fixed grid.
Object tracks	$32 \times 8 \times 3$	Farthest-point samples in the current target mask, with displacement and visibility. Padded points are invalid. The Kabsch variant fits SE(3) to the predicted tracks.
Object DINO	8×1024	Mask-weighted pooling of future patch features with channel layer normalization. Requires a positive mask area.
Scene tracks (2D)	$196 \times 8 \times 3$	Grid source points with two image displacements divided by 224 and a visibility logit.
Dense 2D displacement	$8 \times 2 \times 224 \times 224$	Fixed source pixels, displacement divided by 224. Requires a valid source and a finite projection.
Codec vectors	$8 \times 2 \times 64 \times 64$	Each future/current pair is encoded as P/I frames, and vectors are divided by 64. Only valid vector regions count.
Video-VAE latent	$48 \times 2 \times 14 \times 14$	Frozen Wan2.2 VAE posterior mean with the VAE's own normalization. The two groups end at 200 and 400 ms.
Frozen video model	$48 \times 2 \times 14 \times 14$	Latents that a frozen pretrained image-to-video model predicts from the current frame, read as input tokens without supervision.
Future depth	$8 \times 1 \times 224 \times 224$	Metric depth on finite positive pixels.
Future DINO	$8 \times 196 \times 1024$	Frozen DINOv3 ViT-L/16 teacher [33], 14×14 patches, channel layer normalization.
Dynamic RGB	$8 \times 3 \times 224 \times 224$	RGB divided by 255 on surfaces moving more than 1 mm, including the robot.

Role coverage. On the selected representation, the role rows keep all source points and the same computation and change only which role contributes to the flow loss. The last row removes the role embeddings and tests whether an explicit division into roles helps beyond a task-dependent readout.

Resolution and clearance. The 28^2 and 56^2 grids sample the same image more densely, and the object-sampled flow places its points on the context masks instead of a fixed grid. All variants use identical labels, normalization, and loss weights. The path error e_F is the endpoint error of the flow at the 256 held-out context points nearest to the executed path, at the times when the path passes them. Table S12 reports all variants on the clearance sweep of Appendix B.3, together with a video-latent future at matched capacity.

Extrapolation baselines. Constant-velocity and constant-acceleration forecasts use only the causal SE(3) history visible to the model, i.e., the velocity of the last interval and the difference of the last two interval velocities, applied to translation and, through the rotation vector, to rotation. Offline, we score all three forecasts against the ground truth at the application times. To measure their closed-loop value, we train two variants with the same data and budget (Table S13). The first receives the extrapolated target trajectory as additional input tokens and keeps the world supervision of the selected model. The second replaces the world supervision by these input tokens and is the CV-input baseline of Table 2.

Forecast errors. Physical errors use a common pool of 256 target and 256 context source points, sampled deterministically and excluded from the training queries. Translation and rotation errors keep their metric and geodesic units, and flow errors are endpoint errors on valid correspondences. All forecast errors are measured on held-out episodes at the application times.

D.4. Act

Backbones. The shared-attention backbone shares the attention projections across all token groups and splits the feed-forward networks by group. The separate-group backbone has world and action parameter groups with their own attention projections, adaptive layer normalization, and feed-forward networks, and condition and world tokens share the world group. We match the number of trainable parameters through the feed-forward ratio of the separate-group backbone (3 instead of 4), which leaves both at about 47.3M parameters, with the same depth, width, and number of heads. Equal training budgets do not imply equal FLOPs, so we

Table S11. **Predict: complete future-representation comparison** on held-out scenes (one training seed, three for rows that also appear in Table 3). History and coupling are fixed at the selected configuration, the world stream has the same capacity in every row, and only the world targets change (definitions in Table S10). Rows above the last block keep the selected training-only appearance targets. The last block varies these targets on the selected representation, with their heads dropped at deployment. Pose: translation (mm) and rotation (°) error at 400 ms. e_F: path error of the flow on held-out context points along executed paths (mm). e_t: contact-time error (ms). Cost: world-stream time per query (ms).

Target	Context	SR	SR_{150}	Pose (mm/°)	e_F	e_t	Collision	Cost
none	none	—	—	–	–	—	—	0
SE(3)	none	—	—	—	–	—	—	—
none	flow, 14^2	—	—	–	—	—	—	—
SE(3)	flow, $14^{2\star}$	—	—	—	—	—	—	—
SE(3)	flow, 28^2	—	—	—	—	—	—	—
SE(3)	flow, 56^2	—	—	—	—	—	—	—
SE(3)	flow, object-sampled	—	—	—	—	—	—	—
object tracks	flow, 14^2	—	—	–	—	—	—	—
tracks → SE(3) fit	flow, 14^2	—	—	—	—	—	—	—
object DINO	flow, 14^2	—	—	–	—	—	—	—
SE(3)	scene tracks (2D)	—	—	—	–	—	—	—
SE(3)	dense 2D displacement	—	—	—	–	—	—	—
SE(3)	codec motion vectors	—	—	—	–	—	—	—
SE(3)	video-VAE latent	—	—	—	–	—	—	—
SE(3)	frozen video model	—	—	—	–	—	—	—
SE(3)	future depth	—	—	—	–	—	—	—
SE(3)	future DINO	—	—	—	–	—	—	—
SE(3)	dynamic RGB	—	—	—	–	—	—	—
Role coverage on the selected representation								
SE(3)	flow, target points only	—	—	—	—	—	—	—
SE(3)	flow, context points only	—	—	—	—	—	—	—
SE(3)	flow, no role embedding	—	—	—	—	—	—	—
Training-only appearance targets on the selected representation (SE(3) + flow)								
none		—	—	—	—	—	—	—
future depth		—	—	—	—	—	—	—
future DINO features		—	—	—	—	—	—	—
moving-region RGB		—	—	—	—	—	—	—
all three*		—	—	—	—	—	—	—
all three, no current-target RGB reconstruction		—	—	—	—	—	—	—

Table S12. **Context resolution versus clearance** on the passage tasks: grid spacing at the workspace and path error e_F (mm), and SR (%) at the swept clearances $c_1 < \cdots < c_5$ (Appendix B.3). The target is predicted in SE(3) throughout.

Context	Spacing	e_F	c_1	c_2	c_3	c_4	c_5
none	–	–	—	—	—	—	—
flow, 14^2	—	—	—	—	—	—	—
flow, 28^2	—	—	—	—	—	—	—
flow, 56^2	—	—	—	—	—	—	—
flow, object-sampled	—	—	—	—	—	—	—
video-VAE latent	–	–	—	—	—	—	—

report the measured costs.

Couplings. Appendix C.5 defines the attention mask of each coupling. All couplings share the modules, initialization, losses, and training budget. R0 keeps the world queries and heads but disables every world loss, so it matches the capacity of R1 without world supervision. R1 adds the world supervision with every edge from the world to the action closed. We deploy its checkpoint both action-only and with a forecast that is computed but not read, which isolates the cost of an unused forecast. R2 lets the action read the hidden states of the world queries, R3 lets the world read the noisy actions, and R4 opens both directions. R3 and R4 recompute the world at every Euler step. R5 decodes the

Table S13. **World supervision versus extrapolated inputs**, by motion type. Top: SR (%) of variants of SE(3)-WAM, trained with the same data and budget, that receive the constant-velocity (CV) or constant-acceleration (CA) extrapolation of the causal SE(3) history as input tokens, with or without the world supervision. The CV row without world supervision is the CV-input baseline of Table 2. Bottom: target position error at 400 ms (mm) of the two extrapolations and of the learned world head.

	Const. vel.	Harmonic	Roll/slide	Impact
Without world supervision				
no forecast input	—	—	—	—
CV input	—	—	—	—
CA input	—	—	—	—
With world supervision				
SE(3)-WAM (no input)	—	—	—	—
CV input	—	—	—	—
CA input	—	—	—	—
Position error at 400 ms (mm)				
CV extrapolation	—	—	—	—
CA extrapolation	—	—	—	—
learned world head	—	—	—	—

world's explicit geometry and re-encodes it for the action with a geometry encoder of 0.37M parameters, trained end to end through the predicted geometry without teacher forcing. Because R5 changes both what the action reads and where it enters, the comparison of R2 and R5 is between complete designs rather than one factor.

On development scenes, no coupling beat R1 consistently. Letting the world read the action (R3) had the highest mean of three seeds, [+0.6] points over R1, but the whole difference came from the wire passage, and the effects of both directions of access changed sign across seeds. We kept R1, which is also the cheapest. The world supervision itself, R1 against R0, added [3.1] points on average, but its gain ranged from [−1.3] to [+9.6] points across seeds.

Forecast interventions. The interventions hold the condition tokens and the action noise fixed and change only what the action reads from the world. On R2, they replace the keys and values that the action tokens read at every layer. A matched-current swap substitutes the world states of a different development episode whose current target pose is within 5 mm and 5°, and a time shift recomputes them with every horizon offset shifted by 50 ms. On R5, they edit the decoded geometry before it is encoded. Zero motion uses zero 3D displacement and identity target poses with unchanged visibility, a matched-current shuffle takes the geometry of another development episode under the same matching, time shifts resample the trajectory at $\delta_h \pm 50$ ms, and the oracle uses the measured exogenous motion before the first contact. After contact, future labels depend on the action, so the oracle analysis forks a simulator snapshot under a fixed sequence of candidate actions. Identity controls copy the original states, and couplings in which the action reads nothing from the world serve as negative controls.

Table S14. **Act: complete coupling comparison** on held-out scenes. History and future representation are fixed at the selected configuration. The couplings of Appendix D.4 are compared on the selected backbone, and the other backbone is checked with two of them. SR_{static}: stationary-target variants. NFE: network evaluations per query (world/action). Memory: peak GPU memory (GiB). p95 on one RTX 5090.

Backbone	Coupling	SR	SR_{150}	SR_{static}	LIBERO	NFE	p95 (ms)	Memory	e_t
Shared attention*	R0 (world losses off)	—	—	—	—	0/5	—	—	—
	R1 (aux., action only)*	—	—	—	—	0/5	—	—	—
	R1 (aux., forecast computed)	—	—	—	—	1/5	—	—	—
	R2 (world→action, hidden)	—	—	—	—	1/5	—	—	—
	R3 (action→world)	—	—	—	—	5/5	—	—	—
	R4 (bidirectional)	—	—	—	—	5/5	—	—	—
	R5 (decoded geometry)	—	—	—	—	1/5	—	—	—
Separate groups	R2 (world→action)	—	—	—	—	1/5	—	—	—
	R1 (aux., action only)	—	—	—	—	0/5	—	—	—

E. Lifting Real Robot Data to 4D and Pretraining

E.1. Data, Partitions, and Time

The pretraining corpus is DROID [20], recorded at 15 Hz with two exterior cameras and one wrist camera, together with the 3D point tracks and optimized camera calibrations that PointWorld released for it [17]. We associate each released clip with its raw DROID episode by requiring exact agreement of the joint sequence, end-effector poses, gripper states, commands, and camera identity, and we exclude clips whose capture clock is not monotonic or whose association is ambiguous. The model sees the first exterior camera. The training unit is a causal observation window with its aligned future and action labels. We assign episodes to training, development, and held-out partitions (90/5/5, with a fixed random seed) by their original identifier before any filtering, so overlapping clips of an episode cannot cross partitions. Table S15 reports the yield of every stage and counts episodes, windows, actions, and valid labels separately.

Time. All times are capture timestamps, and we do not relabel frames to a nominal rate. The history offsets lie on a 50 ms grid that the 15 Hz frames do not hit, so we resample the causal inputs at the offsets exactly as at deployment (Appendix C.1), between estimates that had been emitted by the query time and without extrapolation. Target labels interpolate the offline trajectory in the same way, linearly in position and along the geodesic in rotation, and not across gaps longer than two source periods. They therefore lie on the horizons of the main model. Flow labels keep PointWorld's own record times, [0.13] to [1.13] s ahead, without interpolation. The world readouts embed each horizon's offset δ_h in physical time (Appendix C.3), so the scene queries are supervised at these offsets during pretraining and on the 50 ms grid afterwards.

Table S15. **DROID lifted to 4D.** Yield of each stage. Windows are causal observation windows with aligned labels. Exogenous windows show target motion before the first robot–target contact (Appendix E.3). The last two rows compare all windows with valid windows.

	Train	Dev.	Held-out
Episodes with PointWorld tracks	—	—	—
associated to raw DROID	—	—	—
one rigid target named and boxed	—	—	—
Windows	—	—	—
exogenous (%)	—	—	—
with causal pose input	—	—	—
Valid actions (%)	—	—	—
Valid SE(3) labels (horizons, %)	—	—	—
Valid 3D flow labels (points, %)	—	—	—
Speed, all / valid (cm/s)	—/—	—/—	—/—
Occluded, all / valid (%)	—/—	—/—	—/—

Actions. We express robot states and Cartesian commands in the robot base frame after the calibrated flange-to-TCP transform. Command labels follow Appendix C.4. Each label is the increment from the pose commanded for the preceding application time to the pose commanded for the current one, with DROID's commanded poses interpolated to the application times $\tau_t + \ell + j\Delta_a$ of the sampled latency. We admit a label if its values are finite and its reference state was available causally. The robot's execution limits are not an admission rule, because a per-axis limit of 5 cm per command would discard [XX]% of the commands in the fastest quartile of motion and bias the base toward slow demonstrations. We mask only the steps at which DROID's collector issued no command ([3.1]% of window slots). Action and flow validity are independent, so an invalid action does not remove the flow labels of its window.

E.2. Target and Its Pose

DROID provides no object poses, and its instructions name the manipulated object without marking it. We take as the target the object named in the instruction, i.e., the direct object of the main verb extracted by a dependency parser ("the mug" in "put the mug in the sink"), with one persistent instance per episode. Grounding DINO [26] boxes this phrase in the first frame in which it scores at least 0.35, and SAM 2.1 [32] tracks its mask. For every frame with a non-empty mask, we prompt WildDet3D [18] with the tight 2D box, the RGB frame, and the fixed intrinsics. It predicts a metric 3D box from the single image with its own geometry estimate and no external depth, and PointWorld's optimized extrinsics map the box to the base frame. We accept a box when the box centers from the two exterior cameras lie within 5 cm of each other in the base frame. We exclude episodes whose instruction names no object, several objects, or a non-rigid one, as well as episodes without an accepted box. An empty mask counts as a missing observation, and we do not fill it with the previous pose.

Body frame. A 3D box defines its frame only up to the symmetries of the box, and the detector may permute or flip its axes from one frame to the next. We therefore map each new box to the equivalent frame (an axis permutation and sign flips that preserve its extents) nearest to the previous valid estimate, which fixes one body frame per episode. Rotation steps above 45° between consecutive frames that remain after this step, in [X.X]% of frames, are marked invalid instead of being smoothed. As in simulation, rotation about the symmetry axis of a symmetric target is excluded from the loss (Sec. 4.2).

E.3. Causal Inputs and Offline Labels

Causal inputs. SAM 2.1 (Hiera-small) propagates the mask causally, and WildDet3D lifts every masked frame in full precision, so that the estimate does not depend on the batch composition. Each query processes only the newly available frames and appends its estimate without rewriting earlier ones. Empty masks, rejected boxes, and rotation jumps set the input validity to false, and training uses these naturally occurring missing states without synthetic dropout. The real robot uses the same path for objects without a CAD model or reference views (Appendix C.2).

Offline labels. The labels use the full episode and the same target phrase. SAM 2 propagates the mask forward and backward, and we accept frames whose two masks reach an IoU of at least 0.5. The boxes from both directions enter one robust smoother, a Huber cost with pose residuals scaled by 5 mm and 3° and velocity-change residuals weighted by 0.1. The velocity term is omitted across gaps and across velocity jumps above 0.2 m/s or 1 rad/s, so that abrupt events are preserved. Twenty manifold Gauss–Newton iterations optimize the cost, and frames whose two boxes disagree by more than 20 mm or 15° are invalid. The target labels are $\tilde{\mathbf{p}}_{t+h} - \tilde{\mathbf{p}}_t$ and $\mathrm{Log}(\tilde{R}_{t+h}\tilde{R}_t^\top)^\vee$ within the offline trajectory. Because they are not offline futures minus causal estimates, they do not absorb corrections of the current pose. A horizon is valid only if its reference and future poses are valid and share identity and body frame.

Scene flow. PointWorld's tracks are already expressed in the base frame. For each point of the grid, the label follows the released track whose initial projection lies nearest to the point within its patch, provided that the track is initially visible, has a valid positive depth inside the image, and agrees with the depth at its nearest pixel within 20 mm. Patches without such a track are invalid, and we do not fill them from neighbors. The label is the track's displacement from its initial position at each record time, and its visibility is PointWorld's. An occluded, out-of-image, or depth-invalid future point masks its displacement loss and supervises only its visibility. The selected tracks lie [0.5] px from the patch center at the median ([9] px at the 95th percentile).

Contact and exogenous motion. The first robot–target contact is the first frame at which the target box comes within 1 cm of the gripper's finger geometry from the robot's kinematics. A window is *exogenous* if the target moves by more than 1 cm or 5° within its 400 ms horizon before that contact, and Table S15 reports their share. The masked-label control of Table S16 removes all world labels after the first contact and keeps the action loss. It retains [XX]% of the low-data gain of pretraining, so [most of this gain does not come from predicting how the robot moves the target].

Validity bias. PointWorld's tracks cover static surfaces more completely than fast-moving ones, and accepted boxes favor unoccluded, well-textured targets. Table S15 compares the target speed and the occlusion between all windows and valid windows. The label-source control of Table S16 replaces the tracks by flow lifted from 2D correspondences [30] with monocular metric depth [41], which covers more of the image.

E.4. Label Quality

We checked the labels on forty clips, fourteen for development and twenty-six held out, with five anchors each. The clips cover seven strata: stationary targets, smooth translation, rotation, contact or slip, occlusion and recovery, weak appearance, and symmetric or articulated parts. The held-out clips were not used to tune the estimators. In a manual check of [200] random episodes, the accepted box lies on the intended target in [XX]% of them. Prefix truncation and future

Figure placeholder: Fig. S2 — lifting a real robot episode to metric 4D (schematic)

Left (shared setup): the instruction with its object phrase underlined ("put the mug in the sink") and the Grounding DINO box on the first frame of both exterior views, whose WildDet3D box centers agree in the base frame; a small inset shows PointWorld's optimized camera pose next to the raw one. **Top row (causal input path, blue):** frames arrive left to right; SAM 2 propagates the mask forward and WildDet3D lifts each masked frame to a metric 3D box; emitted poses are triads stamped with their availability time; an empty mask shows an explicit "missing" token. No arrow points backwards in time. **Bottom row (offline label path, orange):** masks propagated forward and backward, per-frame boxes mapped to one body frame, the two pose tracks fused by the robust smoother with an abrupt event preserved (the grasp), cumulative SE(3) labels $(\mathbf{d}_h, \phi_h)$ on the 50 ms horizons; a second branch picks, in each grid patch, the PointWorld track nearest the patch center and draws its 3D displacement at PointWorld's record times, with invalid points greyed out. A vertical dashed line marks the first robot–target contact, after which the masked-label control drops world labels. **Right:** one training sample = causal inputs (top) + masked labels (bottom). Use one real DROID frame sequence, vector overlays only.

Figure S2. Lifting DROID to metric 4D supervision. The object named in the instruction is boxed and tracked. The inputs reproduce deployment with forward-only tracking, whereas the labels use the whole episode, with bidirectional tracking and smoothing, and PointWorld's 3D tracks. Labels do not rewrite inputs, and invalid labels mask their losses.

Table S16. **Controlled comparison of initializations.** Each row post-trains the simulation model from its own initialization on 10/25/50/100% of the Dynamic-14 demonstrations (held-out SR, %, three post-training seeds). The generic initialization is the one SE(3)-WAM uses. The last columns give each base's forecast error on held-out DROID episodes at the last horizon, for target translation (mm) and rotation (°) against the smoothed labels and for the flow endpoint (mm) against PointWorld tracks. Control bases are pretrained at pilot scale against a world–action base at the same budget (Appendix E.5).

	Dynamic-14 SR, fraction of demonstrations				DROID held-out		
Initialization	10%	25%	50%	100%	mm	°	flow (mm)
Generic visual only (SE(3)-WAM)	—	—	—	—	–	–	–
Action-only base, same episodes	—	—	—	—	–	–	–
World–action base, flow from monocular depth	—	—	—	—	—	—	—
World–action base, world labels masked after contact	—	—	—	—	—	—	—
World–action base, pilot scale	—	—	—	—	—	—	—
World–action base (real-robot base)	—	—	—	—	—	—	—

replacement confirm that emitted inputs, validity, identity, and reference geometry do not change with future frames. Independent manual landmarks and multi-view geometry provide references where they are sufficiently constrained, and we mark physical accuracy as unverified elsewhere. On the held-out clips, the median translation and rotation errors are [X] mm and [X]° for the causal path and [X] mm and [X]° for the offline path. Identity failures occur on [X]% of the anchors, [X]% of the accepted windows are false accepts, and the labels cover [XX]% of the windows. The check took [X] hours of annotation, and lifting all of DROID took [X] GPU-hours. On surfaces that are static in the image, the selected PointWorld tracks move by [0.9] and [5.8] mm (median and 95th percentile) over the horizons, against [18] and [72] mm for flow lifted with monocular depth, whose scale drifts over time. On the moving subset, where neither source has a reference, we report the agreement of both with a forward–backward-consistent 2D tracker.

E.5. Recipe and Scale

Simulation model. SE(3)-WAM trains on Dynamic-14 from the generic visual initialization with the recipe of Table S6. We chose its scale and recipe with a sweep at equal numbers of training samples over two backbone sizes (the reference and about twice its parameters, by depth at fixed width), two global batch sizes (B_0 and four times B_0), and two learning rates per batch size ($\{0.5, 1\}\eta_0$ at B_0 and $\{1, 2\}\eta_0$ at the larger batch), followed by [four] local refinements around the best configuration of each size, all selected on development scenes (Fig. S3a).

DROID base. The base uses the same architecture and initialization and only real data. We trained two pilot sizes, about two and four times the simulation backbone, for one pass on a stratified quarter of the training windows. We selected the size by its held-out forecast and action errors, under the deployment budget on one RTX 5090 (p95 time-to-action of at most 200 ms and no call above 250 ms). The selected size ([XX]M backbone parameters) is pretrained once on all training windows for [four] passes at a global batch size of [256], with AdamW, 2.5% warm-up, and cosine decay to 0.1 of the peak learning rate. The checkpoint with the lowest development loss is the base (Fig. S3b). We train the action-only, label-source, and masked-label controls at the smaller pilot size and budget and compare them with a world–action base of that size (Table S16).

Figure placeholder: Fig. S3 — recipe and scale

(a) Simulation sweep. y: development SR; x groups: the two backbone sizes; within a group, four bars for $(B_0, 0.5\eta_0)$, (B_0, η_0), $(4B_0, \eta_0)$, $(4B_0, 2\eta_0)$ at equal sample exposure; refinements as small diamonds; the selected configuration starred. **(b) DROID base.** x: training samples (log); y: held-out target translation error at the last horizon (left axis, mm) and flow endpoint error (right axis, mm); thin lines for the two pilot sizes, thick for the main run; marker at the selected checkpoint; dotted horizontal line: constant-velocity extrapolation of the same history.

Figure S3. Recipe and scale of the simulation model (a) and the DROID base (b).

E.6. Independent Post-Training and Controls

The real-robot policy is initialized from the selected base and fine-tuned on the conveyor demonstrations only. The base was pretrained on detector poses, whereas the conveyor policy receives poses from the model-based tracker; fine-tuning adapts the history encoder to the tracker, and we did not measure the cost of this change of pose source separately. Simulation weights do not initialize any real-robot policy, and the base does not see simulation data. Each policy creates its own optimizer, schedule, training-only normalization, and readouts, and we re-initialize incompatible action or state projections with the same seed in every control. The reference without the base is fine-tuned from the generic visual initialization on the same demonstrations, with the same search over [three] learning rates and two training lengths on development trials and the same evaluation. The controlled simulation comparison post-trains the base on Dynamic-14 with the downstream recipe and the same data fractions as the generic initialization. The action-only control pretrains R0 on the same windows, so that its world queries receive no objective. The label-source control keeps everything except the flow labels, and the masked-label control removes world labels after the first contact (Table S16).

F. Baselines, External Benchmarks, Real Robot, and Timing

F.1. Baselines

Common conditions. Table S17 summarizes the adaptations. Each baseline keeps its official architecture, inputs, action chunk, and initialization. ACT and Diffusion Policy train from scratch, $\pi_{0.5}$ is fine-tuned from `pi05_base`, and the other methods start from their released weights. All of them train on the same 11,200 demonstrations, whose commands we map to each action space through the same Cartesian controller. Dynamic-14 renders a 224×224 wrist camera in addition to the agent view, and methods whose recipes use a wrist view receive it. SE(3)-WAM and the kinematic baselines use the agent view only. All methods run under the shared executor and the real-time protocol of Appendix B.3, and Table S23 adds their original executors.

Training. Each baseline follows its official recipe for the published number of updates and is trained once, with seed 0. We changed only the data loading and the action space. For DreamVLA, we ran both its original 40-epoch schedule and a 20-epoch cosine schedule and report the one that was better on development scenes. The configuration of SE(3)-WAM, in contrast, was selected by the design study of Appendix D, a search that the baselines do not receive, and its variants use three training seeds.

Inputs. PUMA [12] receives its historical optical flow, computed on the rendered frames. DreamVLA receives seven causal RGB frames, the end-effector position and orientation, the finger positions, and a fixed task text, and it samples with ten DDIM steps and classifier-free guidance of 1.5. OpenWAM-α receives the current RGB views, joint values, gripper opening, and task text, and it generates video and actions jointly with video decoding disabled. DynamicWAM receives its flow history and kinematic descriptors.

SE(3)-augmented baselines. $\pi_{0.5}$+SE(3) and DynamicWAM+SE(3) receive the eight motion tokens of Appendix C.1 from the same causal tracker, with the same validity flags and missing-state embedding. A two-layer MLP maps the tokens to the model's width. For $\pi_{0.5}$, we append them to the prefix after the image tokens, where the action expert attends to them, and for DynamicWAM they join its kinematic descriptors. The output layer of the MLP is zero-initialized, so training starts from the behavior of the base model, which also provides the recipe.

Kinematic baselines. *SE(3)-WAM, CV input* shares the tracker, the condition tokens, and the action stream of SE(3)-WAM but has no world supervision. Instead, the action stream receives the constant-velocity extrapolation of the tracked pose at the application times as input tokens, and the model is trained on the same data with the same recipe. Appendix D.3 adds a variant that receives these tokens and keeps the world supervision. The *Kalman interceptor* has no learned components. A constant-acceleration Kalman filter [4] runs on the tracked pose, with its process noise tuned on development scenes, and predicts the target's trajectory. The interceptor chooses the earliest point of this trajectory that the arm can reach under its velocity limits after the latency budget, approaches it along the expert's pre-grasp offset, and closes, releases, or aligns at the predicted time. It replans at every query and uses the expert's task-specific grasp, placement, and entry frames.

Table S17. **Baseline adaptations.** All methods train on the same 11,200 demonstrations with the inputs of their papers and run under the shared executor of Appendix B.3. Updates × batch follows the official recipe. Exec.: the execution scheme of the original paper (commands executed per chunk before replanning), used only for the original-executor control of Table S23. Params: trainable parameters, frozen encoders excluded. Seeds: training seeds.

Method	Initialization	Inputs	Updates × batch	Exec.	Params	Seeds
ACT [58]	scratch	agent + wrist RGB, robot state	50,000 × 32	temporal agg.	[X]	1
Diffusion Policy [10]	scratch	agent + wrist RGB (published horizon), robot state	50,000 × 32	8	[X]	1
$\pi_{0.5}$ [31]	`pi05_base`	current agent + wrist RGB, robot state, task text	30,000 × 256	10	[X]	1
$\pi_{0.5}$ + SE(3)	`pi05_base`	as $\pi_{0.5}$ + SE(3) motion tokens	30,000 × 256	10	[X]	1
DynamicVLA [46]	released DOM ckpt.	frames $[-2, 0]$, robot state, task text	50,000 × 44	streaming	[X]	1
PUMA [12]	official checkpoint	RGB, optical-flow history, robot state, task text	[X]×[X]	[X]	[X]	1
DreamVLA [55]	released components	seven causal agent + wrist RGB frames, robot state, task text	48,220 × 512	1	[X]	1
OpenWAM-α [42]	official foundation	current agent + wrist RGB, joints, gripper, task text	34,490 × 192	32	[X]	1
DynamicWAM [29]	official initialization	RGB, flow history, kinematic descriptors	40,000 × 128	16	[X]	1
DynamicWAM + SE(3)	official initialization	as DynamicWAM + SE(3) motion tokens	40,000 × 128	16	[X]	1
SE(3)-WAM, CV input	as SE(3)-WAM	as SE(3)-WAM, plus constant-velocity forecast tokens, no world supervision	as SE(3)-WAM	–	[X]	3
Kalman interceptor	none	tracked SE(3) history, expert grasp frames	–	–	–	–
SE(3)-WAM	frozen MAE encoder	current agent RGB-D, SE(3) history, robot state	40,104 × 16	–	62.2M	3

F.2. DOMINO and LIBERO

DOMINO [12] contains 35 tasks of two interaction types, interception and tracking, at three dynamics levels: constant velocity, polynomial trajectories, and stochastic segments with abrupt changes. We use its [N] single-arm tasks on the Franka embodiment in the clean setting, at a speed coefficient of [0.1] and at all three levels, with the official demonstrations, the benchmark's own evaluator, and 100 closed-loop episodes per task and training seed. We report SR and DOMINO's manipulation score (MS), which scales route completion by penalties for leaving the workspace or the view and for collisions. The DOMINO column of Table 2 is the task-balanced SR. On LIBERO [25], we use the Spatial, Object, Goal, and Long suites with ten tasks each, 50 evaluation episodes per task, three training seeds for our variants and one for baselines, and the released demonstrations and initial states. On both benchmarks, SE(3)-WAM trains from the same generic initialization as on Dynamic-14, with domain-specific normalization. The tracker's object model is the simulator mesh of the object that the instruction acts on first, and queries without a valid pose ([XX]% on LIBERO and [XX]% on DOMINO) use the missing-state embedding of Appendix C.1. The mesh is privileged information that the baselines do not receive, so Table S18 also reports SE(3)-WAM with detector poses. It reports the levels and suites separately.

F.3. Real Robot

Platform. The robot is a seven-DoF Franka arm with a parallel-jaw gripper, commanded through a timestamped Cartesian interface at 20 Hz (Fig. S4). A fixed [RealSense D435] records RGB-D at [640 × 480] and [30] Hz, and we calibrate its extrinsics with a checkerboard before each session. SE(3)-WAM uses this third-person view, and $\pi_{0.5}$ also receives the image of a second camera on the wrist, as its recipe requires.

Figure placeholder: Fig. S4 — real-robot setup

Left: photo of the cell: Franka arm with gripper, fixed third-person RGB-D camera, wrist camera, and the belt conveyor, with the base frame and the graspable stretch of the belt drawn. **Right:** the three objects (box with unequal sides, slender can, cup with one handle), training and test instances side by side, each with its equivalent grasps marked; below, one frame per belt speed (0.04, 0.08, 0.12 m/s) at the instant of the first command, annotated with $\|\mathbf{v}\|\Delta$ for the measured latency.

Figure S4. Real-robot setup: the conveyor cell and the three objects with their training and test instances.

Task and success. A belt conveyor carries the object past the arm. A trial succeeds if both fingers hold the object stably within the graspable stretch of the belt and the object is lifted off the belt and held for at least 0.5 s. Touching or pushing the object, a single-finger hold, and a grasp after the object has left the workspace count as failures.

Objects. The three objects differ in how much their orientation matters: a light cardboard box with unequal sides, whose symmetry leaves a few equivalent grasps, a slender empty drink can, which is nearly axisymmetric, and a light plastic cup with one handle. We record the mass, size, material, center of mass, and equivalent grasps of every instance.

Table S18. **External benchmarks** (three training seeds for our variants, one for baselines). DOMINO: SR (%) of the single-arm tasks by interaction type and by dynamics level (L1 constant velocity, L2 polynomial, L3 stochastic with abrupt changes), and the manipulation score MS. LIBERO: SR (%) per suite.

	DOMINO						LIBERO				
Method	Intercept	Track	L1	L2	L3	MS	Spatial	Object	Goal	Long	Mean
ACT	—	—	—	—	—	—	—	—	—	—	—
Diffusion Policy	—	—	—	—	—	—	—	—	—	—	—
$\pi_{0.5}$	—	—	—	—	—	—	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	—	—	—	—	—	—	—	—	—	—	—
DynamicVLA	—	—	—	—	—	—	—	—	—	—	—
PUMA	—	—	—	—	—	—	—	—	—	—	—
DreamVLA	—	—	—	—	—	—	—	—	—	—	—
OpenWAM-α	—	—	—	—	—	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—	—	—	—	—	—
DynamicWAM + SE(3)	—	—	—	—	—	—	—	—	—	—	—
SE(3)-WAM, CV input	—	—	—	—	—	—	—	—	—	—	—
SE(3)-WAM, detector poses	—	—	—	—	—	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—	—	—	—	—	—

Test trials use instances that appear in no demonstration, and demonstrations, development trials, and test trials come from separate sessions.

Speeds. The belt runs at 0.04, 0.08, or 0.12 m/s. We measure its speed with an encoder at the start and end of every block, because the belt slows by up to [3]% over a session as its motor warms up. Trials whose speed deviates from the nominal value by more than 5% are repeated and reported as equipment failures. The gripper needs about [XX] ms to close on the can, and this time is part of the actuation delay in the latency budget of Appendix C.7.

Trials. Each method runs 180 trials, 20 for every object and speed. All methods use the same pre-generated trial keys, which fix the object instance, the speed, and the start position and orientation, sampled from the calibrated graspable region. An external fixture that the policy does not see sets and records the start pose. The keys form three balanced blocks of 60 trials. Each block runs in its own random order, so that drift of the apparatus does not align with the methods, and each fine-tuning seed of a method runs one block. An operator resets the scene by hand. A trial starts at the first usable observation, without a free observation period, and the robot may move at any time. Human contact, safety stops, and manual completion count as failures. Infrastructure failures before the first usable observation are repeated and reported.

Fine-tuning. SE(3)-WAM is fine-tuned from the DROID base on the task's [300] teleoperated demonstrations with three seeds (Appendix E.6). The reference without the base starts from the generic initialization with the same demonstrations, seeds, and search, and $\pi_{0.5}$ is fine-tuned from `pi05_base` on the same demonstrations with three seeds. The tracker uses [a CAD model of the box and reference views of the can and the cup] (Table S5).

Analysis. We measure the contact-time error against the feasible contact window computed from the belt encoder, and failures without contact remain in the SR denominator. Table S19 reports successes per object, speed, and block, with 95% binomial intervals for the pooled rates and paired differences from a bootstrap over trial keys. We report these results separately from the simulated conveyor task, since physics and cameras differ. The evaluation took [four] days. Most failures of SE(3)-WAM are [late closures on the can, whose pose the tracker loses when the gripper shadows it], and on the cup they are mostly [grasps that close on the handle side].

F.4. Timing

We time all methods on one RTX 5090 (32 GB), the robot's deployment GPU. For each method, timing uses thirty warm-up calls and 300 measured calls over three observation sequences of eight frames, with CUDA synchronization at call boundaries and three rounds with rotated method order, and each method uses the precision of its release. The time-to-action includes tracking the newest frame, encoding the image, and generating the first command, and we record the time to a complete forecast separately. Table 2 gives the p95 values and Table S17 the parameter counts. On the robot, traces record capture, availability, preprocessing, the

Table S19. **Real-robot successes** per object and belt speed (S, M, F: 0.04, 0.08, 0.12 m/s, out of 20 trials each) and per block (out of 60). Each block holds every object and speed and is run by one fine-tuning seed. All: success rate (%) over the 180 trials of a method with its 95% binomial interval. Δ: paired difference from SE(3)-WAM (points) with a bootstrap interval over trial keys.

	Box			Can			Cup			Block				
Method	S	M	F	S	M	F	S	M	F	1	2	3	All	Δ
$\pi_{0.5}$	—	—	—	—	—	—	—	—	—	—	—	—	—	—
SE(3)-WAM, no DROID base	—	—	—	—	—	—	—	—	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—	—	—	—	—	—	—	—	–

Table S20. **Latency breakdown** of one query of SE(3)-WAM on one RTX 5090 at batch size 1 (ms, p50 / p95), and action age on the robot.

Stage	p50	p95
Pose tracking (per frame)	—	—
Image encoding	—	—
Condition tokens (cached)	—	—
Action generation (no world)	—	—
Time-to-action (total)	—	—
Optional forecast (not read)	—	—
Action age at application (robot)	—	—
Deadline misses (robot, %)	—	

start and end of inference, queue entry, command issue, and application. The action age is the application time minus the capture time, and we report its p50, p95, and maximum together with the fraction of missed deadlines and the peak memory (Table S20). The latency sweep of Dynamic-14 differs from wall-clock deployment, because it inserts a prescribed delay while physics advances.

G. Additional Results

G.1. Per-Task Results and Outcomes

Table S21 breaks the held-out Dynamic-14 results of Table 2 down by task for every method, with the paired interval of SE(3)-WAM against the strongest other method on each task. Table S22 reports how episodes end. A method that trades one failure for another, e.g., fewer collisions for more timeouts, shows this only here. Every failure stays in the success-rate denominator.

G.2. Protocols and Latency

Table S23 evaluates every method under three further protocols. Paused physics removes the cost of computation. Equal latency charges every method the p95 latency of SE(3)-WAM and so separates a method's speed from the way it uses time. The original executors test whether our shared executor puts any method at a disadvantage. The ranking depends on the protocol. With paused physics, [DreamVLA] leads SE(3)-WAM by [X.X] points, and under equal latency the difference is [X.X] points. The original executors change the SR of the baselines by at most [X.X] points. Table S24 gives success at every added latency of the sweep, and its rows for SE(3)-WAM and the strongest baseline are the data of Fig. 1 (right).

G.3. Negative and Null Results

Visual histories. None of the eight visual histories of Table S8 improved on the current frame by more than two points. The intervals of [N] of them lay within ±2 points, and [N] were inconclusive, including the RGB-D window with metric positions. Frozen readouts recover past target positions from these histories with [X]–[X] mm error, against [X] mm for the SE(3) history. The information is therefore present, but at our data scale the policy does not turn it into metric motion. A larger dataset or a dedicated motion objective might change this.

Image-space futures. As the world target, no image-space future of Table S11 (scene tracks, dense displacement, codec vectors, video latents, a frozen video model, depth, DINO, or dynamic RGB) exceeded the SE(3) future without context by more than [X.X] points. Their objectives weight pixels or latent dimensions, while the tolerance depends on metric error. Added next to the selected world as training-only targets, depth, DINO features, and moving-region RGB gained [X.X] points. Appearance therefore helps as an auxiliary target of the scene queries, although it does not replace the metric future.

Context resolution. Above the path error of the coarse flow, the 28^2 and 56^2 grids and the object-sampled flow matched the 14^2 grid within two points at up to [X]× its cost. Their advantage is confined to the tightest clearances.

Coupling. Letting the world read the action (R3) or both directions (R4) did not improve consistently over R1 at any latency (Table S14). No coupling in which the action reads the world exceeded the training-only world by more than [X.X] points. Disabling the world losses in the same network (R0) cost [X.X] points against R1, so the gain comes from the supervision and not from the added queries. In R1, no edge leads from the world to the action, so computing the forecast

Table S21. **Per-task held-out success** (%) on Dynamic-14 for every method of Table 2, mean of three training seeds for our variants and one for baselines, real-time protocol. DP: Diffusion Policy. DVLA: DynamicVLA. DrVLA: DreamVLA. OWAM: OpenWAM-α. DWAM: DynamicWAM. +: with our SE(3) history tokens. CV: SE(3)-WAM with CV input. KF: Kalman interceptor. Det.: SE(3)-WAM with detector poses. Δ: SE(3)-WAM minus the strongest other method on that task, with its 95% paired interval. Task codes are listed in Table S3.

Task	ACT	DP	$\pi_{0.5}$	$\pi_{0.5}$+	DVLA	PUMA	DrVLA	OWAM	DWAM	DWAM+	CV	KF	Det.	**Ours**	Δ [95% CI]
Harmonic grasp	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Conveyor grasp	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Translating grasp	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Rotating grasp	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Occluded grasp	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Moving receptacle	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Rotating receptacle	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Wire passage	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Wire passage, full	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Ball capture	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Ball capture, table	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Slider capture	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Can capture	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Post-impact grasp	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Task-balanced	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—

Table S22. **Outcome categories** on held-out Dynamic-14 scenes (% of episodes, seed-averaged, real-time protocol). Categories follow Appendix B.1, and rows sum to 100.

Method	Success	Safety	Task	Bound.	Timeout	Inval.
ACT	—	—	—	—	—	—
Diffusion Policy	—	—	—	—	—	—
$\pi_{0.5}$	—	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	—	—	—	—	—	—
DynamicVLA	—	—	—	—	—	—
PUMA	—	—	—	—	—	—
DreamVLA	—	—	—	—	—	—
OpenWAM-α	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—
DynamicWAM + SE(3)	—	—	—	—	—	—
SE(3)-WAM, CV input	—	—	—	—	—	—
Kalman interceptor	—	—	—	—	—	—
SE(3)-WAM, detector	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—

Table S23. **Protocol controls** on held-out Dynamic-14 scenes (SR, %). Paused: physics stops during inference, and each method uses the execution scheme of its paper. Equal latency: shared executor, with every method charged the p95 latency of SE(3)-WAM. Original executor: real time with each method's own execution scheme. Real time: the protocol of Table 2. Dropped: commands discarded as stale in real time (%).

Method	Paused	Equal latency	Original executor	Real time	Dropped (%)
ACT	—	—	—	—	—
Diffusion Policy	—	—	—	—	—
$\pi_{0.5}$	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	—	—	—	—	—
DynamicVLA	—	—	—	—	—
PUMA	—	—	—	—	—
DreamVLA	—	—	—	—	—
OpenWAM-α	—	—	—	—	—
DynamicWAM	—	—	—	—	—
DynamicWAM + SE(3)	—	—	—	—	—
SE(3)-WAM, CV input	—	—	–	—	—
Kalman interceptor	—	—	–	—	—
SE(3)-WAM, detector poses	—	—	–	—	—
SE(3)-WAM	—	—	–	—	—

at deployment leaves the actions unchanged and only adds [X.X] ms. Under the real-time protocol, this deployment lost [X.X] points against the action-only one.

Unsolved tasks. The SE(3) placement into the rotating receptacle (t2_se3) fails for every method (Table S21). [The policies reach the receptacle but miss its angular tolerance before it turns away.] No variant of SE(3)-WAM solves the full wire passage (t3_full), which adds grasping and releasing the wand to the passage, whereas [OpenWAM-α] succeeds in [XX]% of these episodes. On the passage that starts with the wand grasped (t3_phase), the success of SE(3)-WAM ranges from [0] to [71]% across training seeds.

Other failures. The collapse of Fig. 4a is weakest for the capture tasks (Spearman $\rho = [X.XX]$), where success also depends on when the gripper closes, and the noise crossover lies [X.X] times above the model-free $\sigma^\star$. Post-impact grasp-

Table S24. **Success versus added latency** on held-out Dynamic-14 scenes (SR in %, real-time protocol). The 0 and 150 ms columns equal the All and +150 ms columns of Table 2. Slope: least-squares change of SR per 100 ms of added latency (points).

	Added latency (ms)					
Method	0	50	100	150	200	Slope
ACT	—	—	—	—	—	—
Diffusion Policy	—	—	—	—	—	—
$\pi_{0.5}$	—	—	—	—	—	—
$\pi_{0.5}$ + SE(3)	—	—	—	—	—	—
DynamicVLA	—	—	—	—	—	—
PUMA	—	—	—	—	—	—
DreamVLA	—	—	—	—	—	—
OpenWAM-α	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—
DynamicWAM + SE(3)	—	—	—	—	—	—
SE(3)-WAM, CV input	—	—	—	—	—	—
Kalman interceptor	—	—	—	—	—	—
SE(3)-WAM, detector poses	—	—	—	—	—	—
SE(3)-WAM	—	—	—	—	—	—

ing gains least from world supervision ([X.X] points over the CV input), since a conditional mean cannot represent the outcomes of an impact. Detector poses cost [X.X] points, mostly on symmetric and thin targets. Most remaining failures of SE(3)-WAM are [e.g., timeouts after a missed first grasp] (Table S22).

G.4. Forecasts at Application Time

Fig. S5 aligns the query, the forecast, the ground truth, the issued actions, and the first contact on physical timestamps. The examples show where forecast errors arise—e.g., at the moment of an impact or when a rolling object leaves its ramp—and how they turn into contact-time and pose errors.

G.5. Supplementary Video

The supplementary video shows conveyor trials of SE(3)-WAM and $\pi_{0.5}$ at all three belt speeds with the forecasts overlaid, a set of Dynamic-14 episodes per task family, and the failure cases discussed in Sec. 5.5.

Figure placeholder: Fig. S5 — forecasts aligned with what happened

Rows 1–4: the median-error held-out episode of each motion type (conveyor, rotating target, rolling capture, post-impact); **row 5:** a failure. **Columns:** the query frame at τ_t; the frame at the first command's application time τ^{a}; and a 3D inset in the base frame. **Overlays:** blue past SE(3) poses; orange predicted target triads at $\tau_t + \delta_h$ and coarse context-flow arrows; gray ground-truth poses at the same physical times; purple issued gripper path; a marker at first contact. A strip under each row plots translation and rotation error vs. horizon for learned SE(3), constant velocity, and constant acceleration. Annotate the failure cause of row 5 in red.

Figure S5. Forecasts of the world head on held-out episodes, aligned in physical time with the ground truth and the issued actions. Rows 1–4 show the median-error episode of each motion type, and row 5 shows a failure.